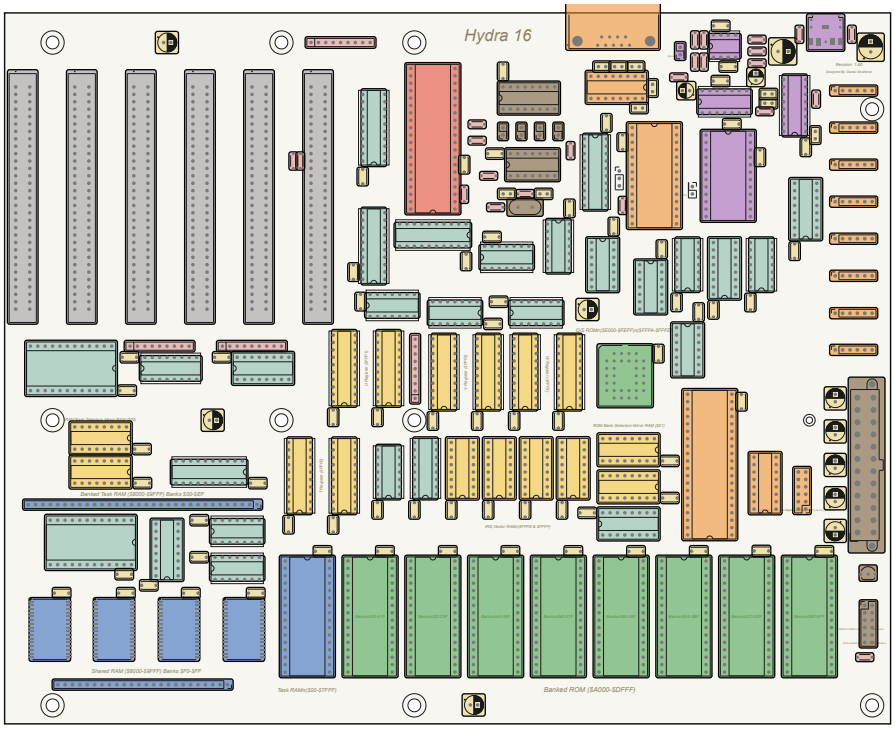
THE HYDRA PROJECT

HYDRA-16

Student Companion



A build log, lab sheets, projects and reference cards
to use with the User's and Builder's Manual

This book belongs to: _____

How to Use This Companion

This is a book to write in. It goes with the *Hydra-16 User's and Builder's Manual*: the manual explains, and this companion is where you do things and keep a record of what you did. Keep the two side by side.

Wherever this companion says **Chapter 6** or **section 6.4**, it means that chapter or section of the manual. Read it first. The companion doesn't repeat what the manual says.

What's in it

Part One, the Build Log, is for builders. It has a box to tick for every group of parts in the manual's Chapters 3 to 5, places to write down what you measured, and a chart for finding faults. If you're using the emulator and not building a Hydra, skip it.

Part Two, the Lab Sheets, is for everyone. Each lab goes with a chapter of the manual, takes one sitting, and ends with a few questions. They work on a real Hydra and on the emulator, except where a lab says otherwise.

Part Three, the Projects, are bigger pieces of work: a program to type in, get going, and then make your own.

Part Four, the Reference Cards, are single sheets to keep open beside the keyboard: the keys, the commands, each language, and the processor's instructions.

Part Five, Your Record, is a logbook, and a list of the skills you've gained, to tick off as you go.

The signs

SIGN	MEANS
☐	A box to tick when you've done something, or checked it
✎	Write your answer on the lines
★	A challenge: harder, and optional
⚠	Take care: read the manual's caution before you go on

Good habits

- **Write what you see, not what you expected.** If the Hydra says something different from the manual, write that down. It's the most interesting thing that can happen in a lab. The manual's section "This book's Hydra" lists the differences that are normal.
- **Date every entry.** A build takes weeks, and you will forget which evening you fitted the capacitors.
- **Save your work where it's kept** (section 6.7). At the end of every lab: is it on the SD card?
- **When you're stuck, write the question down** before you ask for help. Half the time, writing it answers it.

> **NOTE:** There are no answers in this book. Your teacher has them, in the *Educator's Course Guide*. If you're working on your own, the Hydra has them: nearly every question here can be settled by trying it.

Contents

How to Use This Companion 2

PART ONE: THE BUILD LOG

1 Before You Build 7
- The safety rules 7
- The tools 7
- Practice first 8
- Reading the parts 8
- The kit, counted 9
- The plan 9

2 Soldering the Main Board 11
- Step 1: The shared RAM chips 11
- Step 2: The resistors and resistor networks 11
- Step 3: The ceramic capacitors 12
- Step 4: The chip sockets 12
- Step 5: The crystal, the reset chip, the headers and the pin sockets 13
- Step 6: The electrolytic capacitors 13
- Step 7: The big connectors 13
- Step 8: Inspect and clean 14

3 Power, ROMs and Chips 15
- The power test 15
- The ROM chips 15
- The chips 16
- The jumpers 17
- The power, the switch, the reset button and the light 17

4 The Memory Card 19
- The through-hole card 19
- The module number 19
- Plugging it in 20

5 First Switch-On 21
- Before the switch 21
- The serial line 21
- Switching on 21
- If it doesn't start: the fault-finding chart 22
- The fault diary 23
- The hardware test's results 24

PART TWO: LAB SHEETS

1 Meet the Computer 26
2 A Hydra on Your PC 29
3 First Steps 32
4 From Power to Prompt 35
5 Talking to the Shell 38
6 Scripts 41
7 The Tree of Files 44
8 Windows 47
9 A Command Hunt 50
10 Devices Are Files 53
11 Disks 56
12 Sound 59
13 Pictures 62
14 Pins, and the System's Own Files 65
15 Numbers and Bases 68
16 HyForth: The Stack 71
17 HyForth: Decisions, Loops and Programs 74
18 hylang: Expressions and Functions 77
19 hylang: Lists, Decisions and Programs 80
20 HyBASIC: First Programs 83
21 HyBASIC: Bigger Programs 86
22 Your Language Meets the Hydra 89
23 Three Languages 92
24 The 65C02, on Paper 95
25 Your First Machine Code 98
26 Taking Programs Apart 101

PART THREE: PROJECTS

1 A Quiz 105
2 Turtle Pictures 108
3 A Round for Two Voices 111
4 Your Own Command 113
5 A Program in the Background 115
6 A Traffic Light 117
7 Faster, with Machine Code 120

PART FOUR: REFERENCE CARDS

1 Keys 124
2 Everyday Commands 126
3 rc 128
4 HyForth 130
5 hylang 132
6 HyBASIC 134
7 The 65C02 137
8 Numbers and Memory 139

PART FIVE: YOUR RECORD

The Logbook 142
The Skills Passport 145

PART ONE

The Build Log

A place to plan the build, tick off every part as it goes in, write down what you measured, and track down whatever doesn't work the first time. It follows the manual's Chapters 3, 4 and 5, step for step.

A build log isn't busywork. When a Hydra doesn't start, the first question is always "what changed since it last worked?", and the builders who can answer it find their fault in minutes.

- Unit 1. Before You Build
- Unit 2. Soldering the Main Board
- Unit 3. Power, ROMs and Chips
- Unit 4. The Memory Card
- Unit 5. First Switch-On

UNIT 1

Before You Build

THIS STAGE: Manual: Chapter 3, and section 4.1. Do all of this before the soldering iron is switched on. It takes an evening, and it saves a week.

Builder _____ Building with _____ Started on _____

The safety rules

Read section 3.1. Then tick each rule, to say that you understand it and will keep it.

☐	THE RULE
☐	I wear safety glasses whenever I solder or clip leads.
☐	The iron is in its stand whenever it isn't in my hand. I never leave it switched on when I walk away.
☐	I work where the air moves, and I keep my face out of the smoke.
☐	I don't eat or drink at the bench, and I wash my hands when I finish.
☐	Long hair is tied back, and my sleeves are out of the way.
☐	If the iron falls, I let it fall.
☐	The power supply is unplugged whenever I solder, fit a chip or move a jumper.
☐	If I burn myself: cool running water for ten minutes, and I tell an adult.

SIGNED: I've read the manual's section 3.1, and I'll work by these rules. Builder: Adult, teacher or second builder: Date: __

✎ Which solder are you using: lead-free, or solder with lead? What does that change about how you work?

The tools

Tick each tool when it's on the bench (section 3.2).

☐	TOOL	☐	TOOL
☐	Soldering iron with temperature control	☐	Tweezers
☐	Iron stand, with a sponge or brass cleaner	☐	Magnifier or magnifying lamp

☐	TOOL	☐	TOOL
☐	Solder, 0.5 to 0.8 mm, rosin core	☐	Multimeter
☐	Flux pen	☐	Anti-static wrist strap
☐	Desoldering braid, or a solder sucker	☐	IC insertion and extraction tools
☐	Flush cutters	☐	Isopropyl alcohol and an old toothbrush
☐	Needle-nose pliers	☐	Safety glasses
☐	Heat-proof mat	☐	Masking tape, or a piece of foam to hold parts

✎ What temperature is your iron set to?

Practice first

Don't make your first joint on the Hydra. On a scrap board or a practice kit, make joints until they look like the good one in the manual's Figure 3-3, and then make a mistake on purpose and repair it.

☐	PRACTICE	HOW MANY	CHECKED BY
☐	Resistors soldered and their leads clipped	10	
☐	A chip socket soldered: one corner first, checked, then the rest	1	
☐	A solder bridge made on purpose, and cleared with braid	1	
☐	A part taken out again without lifting a pad	1	

✎ Draw a good joint and a bad one, from the side, and say how you can tell them apart.

Reading the parts

Section 3.4 shows how each kind of part is marked. Prove you can read them before you need to.

✎ A resistor has the bands brown, black, orange, gold. What's its value?

✎ A small capacitor is marked 104. What's its value? And one marked 330?

✎ How do you find pin 1 of a chip? Of a socket? Of a header?

✎ Which parts in the kit must go in the right way round? List the kinds.

The kit, counted

Sort the parts into groups, and count each group against the manual's Appendix I. Write down anything missing or extra *before* you start.

☐	GROUP	THE KIT SHOULD HAVE	I COUNTED
☐	Surface-mount RAM chips (may be on the board already)	4	
☐	Resistors	23	
☐	Resistor networks	4	
☐	Ceramic capacitors	78	
☐	Electrolytic capacitors	13	
☐	Chip sockets, of all sizes	64	
☐	Chips for the sockets (five ROM sockets stay empty)	59	
☐	The crystal and the reset chip	2	
☐	Pin headers and pin sockets	19	
☐	Connectors: serial, audio, power and six slots	9	
☐	Memory card: its board and its 13 parts	1	

✎ Missing, extra or damaged:

The plan

The main board has 2,234 joints, and a memory card 240 more. Plan the sessions before you begin: an hour or two each, with a clear place to stop.

SESSION	DATE	WHAT I PLAN TO DO	HOURS PLANNED	HOURS IT TOOK
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

UNIT 2

Soldering the Main Board

THIS STAGE: Manual: section 4.2, with Chapter 3 open beside it. Seven steps of soldering and one of checking: 216 parts and 2,234 joints, in about ten sessions.

For each row: tick **Fitted** when every part in the row is soldered in, and **Checked** when you've looked at every one of its joints with a magnifier (section 3.6). At the end of each step, get someone else to look too. A second pair of eyes finds what yours have stopped seeing.

Step 1: The shared RAM chips

128 joints. Section 3.7 shows how to solder surface-mount chips. If your board came with these already fitted, tick them as checked, and go on.

FITTED	CHECKED	PART	WHAT TO CHECK
☐	☐	U25	Pin 1 at the dot; no bridges between legs; every leg down on its pad
☐	☐	U27	The same
☐	☐	U28	The same
☐	☐	U29	The same

Step 1 finished on _____ Hours _____ Looked over by _____

Step 2: The resistors and resistor networks

82 joints. Resistors go in either way round. A resistor network has a dot at pin 1, which goes to the square pad.

FITTED	CHECKED	VALUE	CODE	PARTS	HOW MANY
☐	☐	22 Ω	220	R23, R24, R25, R26, R27	5
☐	☐	33 Ω	330	R30, R31	2
☐	☐	100 Ω	101	R3	1
☐	☐	330 Ω	331	R22	1
☐	☐	470 Ω	471	R2	1
☐	☐	10 kΩ	103	R4, R5, R6, R7, R13, R19, R20, R21, R28, R32, R33	11
☐	☐	1 MΩ	105	R1	1
☐	☐	0 Ω	0	R29	1

FITTED	CHECKED	VALUE	CODE	PARTS	HOW MANY
☐	☐	3.3 kΩ	332	RN1, RN2, RN3	3
☐	☐	10 kΩ	103	RN4	1

Step 2 finished on _____ Hours _____ Looked over by _____

Step 3: The ceramic capacitors

156 joints. They go in either way round. Most of them are the same value, so do the odd ones first, while they're easy to tell apart.

FITTED	CHECKED	VALUE	CODE	PARTS	HOW MANY
☐	☐	33 pF	330	C47, C48	2
☐	☐	3.3 nF	332	C50, C52	2
☐	☐	100 nF	104	C1, C2, C3, C5, C6, C12 to C46, C57 to C64, C66 to C85	68
☐	☐	1 µF	105	C4, C7, C8, C9, C10	5
☐	☐	4.7 µF	475	C49	1

Step 3 finished on _____ Hours _____ Looked over by _____

Step 4: The chip sockets

1,282 joints: more than half of the board. Every socket has a notch at one end, which must match the notch in its outline on the board. Solder one corner pin, check that the socket is flat and the right way round, then the opposite corner, then the rest.

FITTED	CHECKED	SOCKET	HOW MANY	PLACES
☐	☐	8 pins	1	IC9
☐	☐	14 pins	15	U8, U9, U10, U11, U12, U13, U14, U15, U16, U39, U42, U44, U56, U57, U61
☐	☐	16 pins	19	IC1 to IC8, U4, U5, U17, U18, U20, U24, U26, U40, U41, U45, J28
☐	☐	20 pins	13	U21, U22, U48, U49, U50, U51, U52, U53, U54, U55, U58, U59, U60
☐	☐	24 pins (wide)	3	U19, U23, U38
☐	☐	28 pins (wide)	1	U3
☐	☐	32 pins (wide)	9	U7, U30 to U37
☐	☐	40 pins (wide)	2	U1, U2
☐	☐	PLCC 32 (square)	1	U6

✎ This step takes several sessions. Keep a note of where you stopped each time.

Step 4 finished on _____ Hours _____ Looked over by _____

Step 5: The crystal, the reset chip, the headers and the pin sockets

148 joints.

FITTED	CHECKED	PARTS	WHAT TO CHECK
☐	☐	Y1, the crystal	Soldered quickly; its can not touching any pad
☐	☐	U43, the reset chip	Its flat side matches the outline
☐	☐	J5, J6, J7, J8: the clock jumpers' headers	Standing straight
☐	☐	J9, J29 (2 pins each) and J30 (3 pins)	Standing straight
☐	☐	J10, the front panel, and J27, the I/O header (2 rows of 6 each)	Standing straight; the short pins through the board
☐	☐	J18 to J25: the eight SPI pin sockets	Flat on the board
☐	☐	J1 (32 holes) and J2 (24 holes): the memory card's sockets	Perfectly straight and flat: the card plugs into both at once

Step 5 finished on _____ Hours _____ Looked over by _____

Step 6: The electrolytic capacitors

26 joints. ⚠ **These must go the right way round**, except the two marked NP. The longer leg is plus; the stripe on the can marks minus. One fitted backwards can burst.

FITTED	CHECKED	VALUE	CODE	PARTS	POLARITY	HOW MANY
☐	☐	10 µF	106	C90, C91	Yes	2
☐	☐	47 µF	476	C86, C87, C88, C89	Yes	4
☐	☐	100 µF	107	C51, C53, C54, C55, C56	Yes	5
☐	☐	220 µF non-polarized	227 NP	C92, C93	No: either way	2

☐	BEFORE SOLDERING EACH ONE
☐	I checked every polarized capacitor's plus leg against the + on the board, twice.

Step 6 finished on _____ Hours _____ Looked over by _____

Step 7: The big connectors

412 joints, 372 of them in the six slots.

FITTED	CHECKED	PART	WHAT TO CHECK
☐	☐	J3, the serial connector	Its two big lugs soldered, as well as its nine pins
☐	☐	J26, the audio jack	Flat on the board
☐	☐	J12, slot 0	Its two end pins first; flat and straight; then all 62
☐	☐	J13, slot 1	The same
☐	☐	J14, slot 2	The same
☐	☐	J15, slot 3	The same
☐	☐	J16, slot 4	The same

FITTED	CHECKED	PART	WHAT TO CHECK
☐	☐	J17, slot 5	The same
☐	☐	J11, the power connector	Its latch matches the outline; every joint full and shiny

Step 7 finished on _____ Hours _____ Looked over by _____

Step 8: Inspect and clean

Go over the whole back of the board, slowly, with a magnifier and a good light. Work in strips, and tick each strip when it's done, so that no part of the board is missed.

☐	STRIP OF THE BOARD	JOINTS I REPAIRED
☐	The top quarter	
☐	The second quarter	
☐	The third quarter	
☐	The bottom quarter	
☐	Every electrolytic capacitor's polarity, from the front	
☐	Every socket's notch, from the front	
☐	The flux cleaned off, and the board dry	

✎ What did the inspection find? (Everybody finds something.)

SIGNED: The board has been inspected, front and back, by its builder and by one other person. Builder: Second pair of eyes: Date: __

UNIT 3

Power, ROMs and Chips

THIS STAGE: Manual: sections 4.3 to 4.6, and 4.8. The board gets power for the first time, *before* any chip is in it; then the ROM chips are programmed, and every chip goes into its socket.

The power test

⚠ No chips in the sockets yet. If something is wrong with the power, this is when you want to find out.

Before the power supply is connected (section 4.3):

☐	CHECK	WHAT I MEASURED
☐	Resistance from pin 8 to pin 21 of U1's socket: it should be *more* than about 100 ohms	

✎ If the reading was close to zero, what did you find?

With the power supply and a switch connected (section 4.8), and switched on:

☐	RED PROBE ON	SHOULD READ	WHAT I MEASURED
☐	U1 socket, pin 8	+4.75 to +5.25 V	
☐	U42 socket, pin 4	about +12 V	
☐	U42 socket, pin 11	about –12 V	

☐	AND
☐	Nothing got hot, and nothing smells of burning.
☐	The power light lights (if it's wired).
☐	The power is switched off again, **and unplugged**, before I go on.

The ROM chips

Skip this if your ROM chips came programmed and labelled (section 4.4).

PROGRAMMED	VERIFIED	LABELLED	FILE	CHIP TYPE	GOES IN
☐	☐	☐	`bios.bin`	SST39SF010A (PLCC)	U6

PROGRAMMED	VERIFIED	LABELLED	FILE	CHIP TYPE	GOES IN
☐	☐	☐	`prom0.bin`	SST39SF040	U31
☐	☐	☐	`prom1.bin`	SST39SF040	U32
☐	☐	☐	`prom2.bin`	SST39SF040	U34

✎ Which programmer did you use, on what date were the ROM images built, and from which version of HydraOS? (You'll want to know this when you program them again.)

The chips

⚠ Wrist strap on. Power unplugged. Read the number on each chip, and the number printed on the board beside its socket, before the chip goes in: many of them look alike (section 4.5). The notch on the chip matches the notch on the socket.

FITTED	CHECKED	CHIP	PLACES	HOW MANY
☐	☐	W65C02S (processor)	U1	1
☐	☐	R65C22 (VIA)	U2	1
☐	☐	R65C51 (ACIA, serial)	U3	1
☐	☐	MAX232 (serial levels)	U5	1
☐	☐	SST39SF010A, programmed with `bios.bin`	U6	1
☐	☐	HM628512 (task RAM), or a DS1747 for a real-time clock	U7	1
☐	☐	SST39SF040, programmed with `prom0.bin`, `prom1.bin`, `prom2.bin`	U31, U32, U34	3
☐	☐	YM2151 (sound)	U38	1
☐	☐	YM3012 (sound converter)	U41	1
☐	☐	TL074 (op-amps)	U42	1
☐	☐	NJM4556A (headphone amplifier)	IC9	1
☐	☐	SP720 (protection for the I/O header)	J28	1
☐	☐	74F138	U4, U20, U24	3
☐	☐	74LS260	U8, U9	2
☐	☐	74F20	U10	1
☐	☐	74ACT08	U11	1
☐	☐	74F86	U12	1
☐	☐	74F32	U13, U16, U56	3
☐	☐	74F30	U14	1
☐	☐	74F00	U15, U44	2
☐	☐	74LS148	U17, U18	2
☐	☐	74LS154	U19, U23	2

FITTED	CHECKED	CHIP	PLACES	HOW MANY
☐	☐	74F541	U21, U49, U51, U53, U55, U58, U60	7
☐	☐	74ACT240	U22	1
☐	☐	74F139	U26	1
☐	☐	74ACT14	U39	1
☐	☐	74F191	U40	1
☐	☐	74F157	U45	1
☐	☐	74F573	U48, U50, U52, U54	4
☐	☐	74F02	U57	1
☐	☐	74ACT245	U59	1
☐	☐	74F125	U61	1
☐	☐	74LS219	IC1 to IC8	8

☐	AFTERWARDS
☐	I looked along every chip from the side: no leg is bent under, or outside its socket.
☐	The five empty sockets are U30, U33, U35, U36 and U37, and nothing else is empty.

✎ Does your Hydra have a DS1747 clock chip in U7, or an HM628512? And an R65C51 in U3, or a W65C51N?

The jumpers

☐	JUMPER	WHERE IT GOES
☐	The clock	One jumper only, on **J7** (3.58 MHz)
☐	J30	Pins 1 and 2 (or 2 and 3, for a three-wire serial cable)
☐	J9 and J29	**No** jumpers

The power, the switch, the reset button and the light

☐	CONNECTION	TO
☐	The ATX supply's 24-pin plug	J11
☐	A power switch that stays on	J10, pins 11 and 9
☐	A reset push button	J10, pins 7 and 5
☐	A power light: the LED's longer leg to pin 12, its shorter leg to pin 10	J10, pins 12 and 10

✎ Draw J10 as you've wired it, with pin 1 marked.

UNIT 4

The Memory Card

THIS STAGE: Manual: section 4.7. The through-hole card: 13 parts and 240 joints, an evening's work. The Hydra needs at least one memory card (section 1.7): it's where each task's extra memory is.

The through-hole card

FITTED	CHECKED	PART	WHAT IT IS	WHAT TO CHECK
☐	☐	C1 to C5	100 nF ceramic capacitors (104)	Either way round
☐	☐	U1	74LS139 (or its 16-pin socket)	Its notch matches the outline
☐	☐	IC1 to IC4	Four 32-pin wide sockets	Their notches match the outlines
☐	☐	J3	Pin header, 2 rows of 15	Long pins standing *up*
☐	☐	J1	Pin header, 1 row of 32	Long pins pointing *down*: soldered with the card sitting on the main board
☐	☐	J2	Pin header, 1 row of 24	The same
☐	☐	The four HM628512 RAM chips	In IC1 to IC4	Notches matching; no legs bent

✎ J1 and J2 go in upside down. In your own words, how did you keep them straight while you soldered them?

The module number

☐	CHECK
☐	A jumper is on J3, across its two rows, in column 0 (pins 1 and 2): this is module 0.
☐	If there's a second card or module, it has a *different* number.

✎ The module numbers on your cards:

Plugging it in

☐	CHECK
☐	The power is off and unplugged.
☐	The card's J1 is over the main board's J1, and its J2 over J2.
☐	The card is pushed down evenly, all the way.

✎ When the Hydra starts, it prints `RAM modules:` and a number, in hex. What do you expect yours to print?

Card finished on _____ Hours _____ Looked over by _____

UNIT 5

First Switch-On

THIS STAGE: Manual: section 4.9 and Chapter 5. The moment of truth. If the prompt appears, congratulations. If it doesn't, you're in good company: most home-built computers don't work the first time, and finding out why is where builders learn the most.

Before the switch

☐	CHECK (SECTION 4.9)
☐	Every chip is in the right socket, the right way round, with no legs bent.
☐	The programmed ROM chips are in U6, U31, U32 and U34.
☐	One clock jumper, on J7. The J30 jumper on pins 1 and 2.
☐	The memory card is plugged in, with its module jumper set.
☐	The power supply is plugged into J11, and the power switch into J10.
☐	The SD card module, if there is one, is wired to J18 (section 14.2), with a card in it.

The serial line

☐	CHECK (SECTIONS 5.1 AND 5.2)
☐	The adapter says RS-232, and has a 9-pin plug. It isn't a "TTL" adapter, and there's no null-modem cable.
☐	I know the port's name on my PC.
☐	The terminal program is set to 9600 baud, 8 data bits, no parity, 1 stop bit, flow control XON/XOFF.
☐	The terminal program is running *before* I switch on.

✎ The port's name, and the terminal program you're using:

Switching on

Switch on, and copy down what the Hydra says. The lines below are the ones that matter (section 5.3).

WHAT THE HYDRA PRINTED	MINE SAID
`Hydra-16: kernel` …	

WHAT THE HYDRA PRINTED	MINE SAID
`POST ZP:` …	
`RAM U:` …	
`POST ok`, or `POST found faults:` …	
`RAM modules:` …	
`HydraOS 1.0 for the Hydra-16`	
The prompt, `/>`	

Then two commands (section 5.4):

COMMAND	WHAT MINE SAID
`date`	
`free`	

✎ How long was it from your first joint to your first prompt? How does that compare with your plan in Unit 1?

SIGNED: This Hydra started on ___ Builder: Witness:

If it doesn't start: the fault-finding chart

Always switch off, and unplug, before you touch the board. Change **one thing at a time**, and write down what you changed, in the fault diary on the next page. Section 5.7 has the details of every step here.

WHAT YOU SEE	CHECK, IN THIS ORDER	TICK
No power light	The supply's own switch, and its plug in J11	☐
	The power switch on J10, pins 11 and 9: is it a switch that stays on?	☐
	The voltages, as in Unit 3. No +5 V: unplug, and look for a short	☐
Light on, but nothing in the terminal	The right port, at 9600 baud. Unplug the adapter, plug it in again, and see which port appears	☐
	The cable: straight-through, not null-modem; an RS-232 adapter, not a TTL one	☐
	Press reset. Do the messages come now?	☐
	Move the J30 jumper to pins 2 and 3	☐
	Exactly one clock jumper, on J7	☐
	The chips it needs to start: U1, U6, U3, U5, U39, U40, U43. Right socket? Right way round? No bent legs?	☐

WHAT YOU SEE	CHECK, IN THIS ORDER	TICK
	The address decoder chips: U4, U8 to U16, U19 to U22. Read each number against the board	☐
	The joints around those chips, and around J3	☐
Letters, but nonsense	The terminal's speed: 9600 baud, 8N1	☐
	The clock jumper: on J5 or J6, the serial port runs slow too	☐
`POST found faults`	Write down the whole `POST` and `RAM` lines. Each number that isn't 0 points at a chip: the table in section 5.7 says which	☐
`RAM modules: 00`	The card pushed fully into J1 and J2; a jumper on its J3; its RAM chips the right way round	☐
It starts, but something doesn't work	The hardware test: hold T and press reset, or type `hwtest`; then A for all the tests	☐

The fault diary

One line for each thing you try. Yes, even the ones that didn't help: they're how you know what the fault *isn't*.

DATE	WHAT WAS WRONG	WHAT I CHECKED OR CHANGED	WHAT HAPPENED

✎ When it finally worked: what was the fault, and how could you have caught it sooner?

The hardware test's results

Run the hardware test once your Hydra works, and keep the results. If something stops working months from now, you'll know what "good" looked like (section 5.7).

TEST	RESULT	TEST	RESULT
CPU		BIOS ROM	
T U V W registers		Paged ROM	
Shared RAM		Interrupts	
RAM bank registers		VIA	
Task RAM		Sound chip	
RAM modules		CPU clock	
Serial port		SPI devices	
I2C bus		Slot cards	



PART TWO

Lab Sheets

Twenty-six labs, one sitting each. Every lab goes with a chapter of the manual: read the chapter (or the sections the lab names) first, then do the lab at a Hydra or an emulator.

Each lab has things to **do**, with places to write down what you saw; **questions** to answer afterwards; a list to **check yourself** against; and a **challenge**, for when the rest was easy.

- Labs 1 to 3: meeting the Hydra
- Labs 4 to 9: inside HydraOS
- Labs 10 to 14: devices
- Labs 15 to 23: programming languages
- Labs 24 to 26: machine language

LAB 1

Meet the Computer

THIS LAB: Manual: Chapter 1. About 45 minutes. You need the manual's Figure 1-1 (or a real board to look at), and, for the last part, a Hydra or an emulator.

Name _____ Date _____

Do

1. Find each of these on Figure 1-1, or on the board. Write its reference (like U1) and, in a few words, its job.

PART	REFERENCE	ITS JOB
The processor		
The crystal		
The task RAM		
The kernel ROM		
The VIA		
The serial chip (ACIA)		
The sound chip		
The socket for the Vera X		
Where the power comes in		

2. Fill in the table. Work the answers out on paper (section 1.6).

DECIMAL	BINARY (8 BITS)	HEX
10		
65		
200		
	0001 0000	
	1111 1111	
		$2A

DECIMAL	BINARY (8 BITS)	HEX
		`$80`

3. Now let the Hydra check your work. `calc` can write a number in any base: `-b b` is binary, `-b x` is hex, and a number that starts `#x` or `#b` is read in hex or binary. Try each, and correct your table.

```
/> calc -b b 65
/> calc -b x 200
/> calc '#x2A'
/> calc '#b11111111'
```

Questions

1. ✎ A computer does four kinds of thing: input, processing, memory and output. Give one part of the Hydra for each.

2. ✎ The Hydra is named for a monster with many heads. What are the Hydra-16's sixteen "heads"?

3. ✎ Each task has 32K of memory of its own. How many bytes is that? How much is that for all sixteen tasks, in K?

4. ✎ How many different values can one byte hold? Why that number?

5. ✎ What's the difference between RAM and ROM? Which one holds HydraOS, and why?

6. ✎ One hex digit stands for how many bits? So how many hex digits does it take to write a byte? An address?

Check yourself

☐	I CAN
☐	name the Hydra's main parts, and say what each is for
☐	turn a number from decimal into binary and hex, and back
☐	say what a `$` in front of a number means

CHALLENGE: ★ Write the year you were born in binary and in hex. Then: the Hydra's highest address is `$FFFF`. What is that in decimal? What would the highest address be if addresses had five hex digits?

LAB 2

A Hydra on Your PC

THIS LAB: Manual: Chapter 2, sections 2.1 and 2.2. About 45 minutes. You need a PC with a web browser, and the emulator's page, `hydra-16.html`.

Name _____ Date _____

Do

1. Open `hydra-16.html`. Watch the Hydra start. ✎ What's the last line it prints before it waits for you?

2. Click in the serial console, and type these two lines. ✎ What does each print?

```
/> 2 3 + .
/> ls /
```

3. Find each control, try it, and say what happened.

CONTROL	WHAT HAPPENED WHEN I USED IT
Pause	
The speed menu: **As fast as it goes**	
The **size** menu	
Copy text	
Sound	

4. An experiment: where does a file survive? Make one file on the RAM disk and one on the SD card:

```
/> echo hello >/ram/a.txt
/> echo hello >/sd/0/a.txt
```

Then do each thing in the table, and look with `ls /ram` and `ls /sd/0`. Write **kept** or **gone**. (After each one, make the files again if they've gone.)

AFTER	/RAM/A.TXT	/SD/0/A.TXT
Pressing **Reset**		
Pressing **Power cycle**		
Closing the page, and opening it again		

5. Open **Setup...**. ✎ What hardware does this Hydra have?

6. In Setup, clear the box for the Vera X card, and press **Power cycle with these**. Type `ps`. Then put the Vera X back, power cycle, and type `ps` again. ✎ What's different?

Questions

1. ✎ What is an emulator? In what way is it the *same* as a real Hydra, and in what ways is it different?

2. ✎ From your experiment: where should you keep work that matters? Why?

3. ✎ What's the difference between **Reset** and **Power cycle**?

4. ✎ You want to take your work home, to another computer. Which button do you need, and what does it give you?

Check yourself

☐	I CAN
☐	start the emulator, and type a command into it
☐	reset it, power cycle it, and change its hardware in Setup
☐	say where a file is safe, and where it isn't
☐	save my SD card to a file, and load it again

CHALLENGE: ★ If you have the Hydra's files and Node.js (section 2.3), start the terminal emulator with `node reborn/sim/run.js -i`. Press CTRL-A and then S. What does it show? Then CTRL-A B, type `r`, and then `c`. What were you looking at?

LAB 3

First Steps

THIS LAB: Manual: Chapter 6. About an hour. You need a Hydra at its /> prompt.

Name _____ Date _____

Do

1. Type each command, and write down what the Hydra says.

I TYPED	THE HYDRA SAID
`date`	
`echo Hello, Hydra!`	
`ls /sd/0`	
`2 3 + .`	
`free`	

2. Now make these mistakes on purpose. Write down each message, and what it's telling you.

I TYPED	THE HYDRA SAID	WHICH MEANS
`dat`		
`LS`		
`2 3 plus .`		
`2 3+ .`		
`cat /ram/nothing`		

3. Type `sleep 100`. ✎ How do you get the prompt back without waiting?

4. Type `cat`, with nothing after it. Type a few lines. ✎ What does it do with them? How do you make it stop?

5. Start each program, look at its prompt, and leave it again.

TO START IT	ITS PROMPT	TO LEAVE IT
`rc`		
`basic`		
`hylang`		
`: test` (don't finish it!)		

6. Make a file of your own with the screen editor: `edit /ram/me.txt`. Type three lines about yourself. Save it, and leave the editor. ✎ Which keys saved it? Which left the editor?

7. Show your file with `cat`. Then keep it: make a directory for your work, and copy the file there.

```
/> mkdir /sd/0/work
/> cp /ram/me.txt /sd/0/work/me.txt
/> ls -l /sd/0/work
```

✎ What does `ls -l` tell you about your file?

8. Run `/sd/0/sample/c/chorus`. ✎ What goes wrong the first time, and what puts it right?

Questions

1. ✎ What is a prompt? What does the Hydra's prompt tell you, apart from "I'm ready"?

2. ✎ `echo one >/ram/f` and then `echo two >/ram/f`. What's in the file? What if the second had been `>>`?

3. ✎ Why is `/ram` a bad place to keep your work? Where's a good place, on your Hydra?

4. ✎ You type a command, and the Hydra says `not found`. Give three things that might be wrong.

5. ✎ Where would you look to find out what the command `wc` does?

Check yourself

☐	I CAN
☐	type a command with arguments, and fix a line before I send it
☐	bring back the last line with ↑, and change it
☐	stop a program, and end my typing
☐	tell from the prompt which program is listening, and leave it
☐	make a file with `edit`, look at it, copy it and remove it
☐	keep my work on the SD card

CHALLENGE: ★ In HyForth, define a word that prints your name: `: me ." ..." ;`. In HyBASIC, write a two-line program that prints your name five times. In hylang, make it print your name with `(print ...)`. Which felt most natural?

LAB 4

From Power to Prompt

THIS LAB: Manual: Chapter 7 (sections 7.1 to 7.3 and 7.5; dip into 7.4 for the messages). About 45 minutes. You need a Hydra at its prompt.

Name _____ Date _____

Do

1. The Hydra keeps a copy of everything it printed as it started. Read it again:

```
/> cat /dev/kmesg
```

✎ Copy the line that starts `POST`, and the line that says how many RAM modules were found.

2. Type `ps`, and fill in the table for every task it lists.

TASK	NAME	STATE	WHAT IT'S FOR (SECTIONS 7.4 AND 7.5)

3. ✎ One of the tasks in your table is `ps` itself. Why is it there? Will it be there when `ps` has finished?

4. Start a program that takes a while, and leave it running "in the background", by putting `&` after it. Then look for it, and stop it. (Use the task number that *your* `ps` shows for `sleep`.)

```
/> sleep 60 &
/> ps
/> kill 2
/> ps
```

✎ What state was `sleep` in? Which task was its parent? What happened to it after `kill`?

5. Type `free`. ✎ How much memory can each task have? How many RAM modules does that mean?

6. Type `cat /dev/ticks`, twice, a few seconds apart. ✎ What are the two numbers? About how many ticks is a second?

Questions

1. ✎ The 65C02 can only see 64K at once, but the Hydra has megabytes. How does it reach the rest?

2. ✎ What's a task? How does the Hydra keep sixteen of them from getting in each other's way?

3. ✎ Put these in the order they happen when the Hydra starts: the shell starts; the drivers start; the power-on self test; `init` starts; the processor reads the reset vector.

4. ✎ What's a driver? Name three, and the device each looks after.

5. ✎ What happens if the shell in window 0 ends? Why is that a good idea?

6. ✎ What's the kernel's "tick", and what happens on each one?

Check yourself

☐	I CAN
☐	read the start-up messages, and say what the main ones mean
☐	list the tasks, and say which are drivers
☐	start a program in the background, find it, and stop it
☐	explain, roughly, how one processor runs many programs

CHALLENGE: ★ `mods` lists the pieces of HydraOS that live in the paged ROM, and where each is. How many are there (`mods | wc -l` counts the lines)? Find `forth`, `hylang` and `basic` in the list. Which of the three languages takes the most banks? What might that tell you?

LAB 5

Talking to the Shell

THIS LAB: Manual: Chapter 8, sections 8.1 to 8.10. About an hour. You need a Hydra at its prompt. Type `rc` first: this lab is typed at rc's `%` prompt. (`exit` goes back.)

Name _____ Date _____

Do

1. A variable holds a list of words. Type each line, and write what it prints.

I TYPED	IT PRINTED
`pets=(cat dog fish)`	(nothing)
`echo $pets`	
`echo $#pets`	
`echo $pets(2)`	
`echo I have a $pets(1)`	
`echo 'I have a $pets(1)'`	

2. Send output to a file, add to it, and count its lines:

```
% echo one >/ram/n.txt
% echo two >>/ram/n.txt
% echo three >>/ram/n.txt
% cat /ram/n.txt
% wc -l </ram/n.txt
```

✎ What did `wc -l` say? What does the `<` do?

3. A pipe, `|`, sends one program's output straight into another. ✎ Write what each prints, and what it's counting.

I TYPED	IT PRINTED	WHAT THAT IS
`ls /rom/bin | wc -l`		
`ls /sd/0/sample | wc -l`		

I TYPED	IT PRINTED	WHAT THAT IS
`cat /ram/n.txt | sort`		

4. A command's output can become words in another command. Try it:

```
% echo today is `{date}
% now=`{date}
% echo $#now
```

✎ How many words is `date`'s output? What are they?

5. Every command ends with a **status**: empty for success, something else for failure.

```
% ls /rom
% echo $status
% ls /nowhere
% echo $status
% ls /nowhere >[2]/dev/null || echo not there
% ls /rom >/dev/null && echo there
```

✎ What was `$status` after each `ls`? What do `||` and `&&` do?

6. A pattern stands for all the names that match it: `*` is anything, `?` any one character, and `[a-c]` any one of those. ✎ What does each of these list?

I TYPED	WHAT IT LISTED
`ls /rom/bin/s*`	
`ls /rom/bin/??`	
`ls /rom/bin/[x-z]*`	

Questions

1. ✎ The Hydra has two shells in one. How does it decide whether a line is for HyForth or for rc? How can you make sure a line goes to rc?

2. ✎ What's the difference between `echo $pets` and `echo '$pets'`? When would you want quotes?

3. ✎ `>[2]/dev/null` made an error message disappear. Where did it go? What's the `2`?

4. ✎ When you type `ls`, how does the shell find the program? (Try `echo $path`, and `whatis ls`.)

5. ✎ Write one line that counts how many of the programs in `/rom/bin` have a name starting with `s`.

Check yourself

☐	I CAN
☐	set a variable, and use all of it, part of it, and its length
☐	send a command's output to a file, and read its input from one
☐	join commands with a pipe
☐	test whether a command worked
☐	use `*`, `?` and `[ ]` to name many files at once

CHALLENGE: ★ In one line: list the names in `/rom/bin`, sort them backwards (`sort -r`), and show the first three (`head -3`). What are they? Then, in another line: how many of the names have an `e` in them (`grep e`)? A pipeline can join nine programs at most (section 8.7). Why nine? And how would you do a job that needs ten?

LAB 6

Scripts

THIS LAB: Manual: Chapter 8, sections 8.11 to 8.14, and section 9.7. About an hour. You need a Hydra, and Lab 5 done. Type `rc` first: this lab is typed at rc's `%` prompt.

Name _____ Date _____

Do

1. A `for` loop does something for each word in a list. ✎ What does each of these print?

```
% for(i in 1 2 3) echo number $i
% for(f in /sd/0/sample/*) echo I found $f
```

2. `if` runs a command when another succeeds. `~` is the command that tests whether a word matches a pattern.

```
% name=Sam
% if(~ $name Sam) echo hello Sam
% if(~ $name S*) echo starts with S
% if(~ $name Ann) echo hello Ann
% if not echo you are not Ann
```

✎ Which lines printed something?

3. A function is a new command, made of others. `$1` is its first argument, and `$*` all of them.

```
% fn greet { echo Hello, $1! }
% greet Sam
% fn shout { for(w in $*) echo $w! }
% shout one two three
```

✎ What did `shout` print?

4. A **script** is a file of commands. Make this one with `edit /ram/tour`:

```
#!/bin/rc
# tour - say what's on each disk
for(d in /rom /ram /sd/0) {
        echo In $d there are:
        ls $d
}
```

Run it with `rc /ram/tour`. ✎ What does the first line of the script do? What's the `#` line for?

5. Make your script a command of its own, by copying it into `/bin`. Then run it by its name, from anywhere:

```
% cp /ram/tour /bin/tour
% cd /rom
% tour
```

✎ `/bin` is a union of several directories (section 9.7). Which real directory did your script go into? (Look with `ls /ram/bin`. The sample card has no `bin` directory yet: `mkdir /sd/0/bin` makes one, and it joins `/bin` the next time the Hydra starts. `newns` would join it at once, but it also empties your `/ram`, your script with it: section 9.8.)

6. Scripts take arguments too. Change `tour` so that it tours the directories you name, and all three if you name none. (`$#*` is the number of arguments, and `$*` is the list of them.) ✎ Write your new script here.

Questions

1. ✎ What's the difference between a function and a script? When would you choose each?

2. ✎ In `for(f in /sd/0/sample/*)`, who turned the `*` into a list of names: `for`, `echo`, or the shell?

3. ✎ Your script is in `/ram/bin`. What happens to it when the Hydra is switched off? Where should a script you want to keep go?

4. ✎ What does `while(! ~ $i 0)` mean, word by word? (Section 8.11.)

Check yourself

☐	I CAN
☐	write a `for` loop over a list, or over files
☐	test a word with `~`, and act on the answer
☐	write a function with arguments
☐	write a script, run it, and make it a command

CHALLENGE: ★ Write a script `countdown` that counts down from the number it's given to 1, and then prints "Lift off!". You'll need `while`, and `calc` to take 1 away: `i=`{calc $i - 1}`. Then make it wait a second between numbers.

LAB 7

The Tree of Files

THIS LAB: Manual: Chapter 9. About 45 minutes. You need a Hydra at its `/>` prompt.

Name _____ Date _____

Do

1. Walk around the tree. After each line, write where you are (the prompt tells you, and so does `pwd`).

I TYPED	WHERE I AM NOW
`cd /rom/lib/forth`	
`cd ..`	
`cd ../songs`	
`cd /`	

2. ✎ Draw the part of the tree you just walked through: `/`, `rom`, `lib`, `forth` and `songs`.

3. Type `ls -l /rom`. ✎ How can you tell a directory from a file in a long listing? Which is the biggest file there?

4. Type `df`, and fill in the table.

DISK	WHERE IT APPEARS	ITS SIZE	HOW MUCH IS FREE

DISK	WHERE IT APPEARS	ITS SIZE	HOW MUCH IS FREE

5. Devices look like files. ✎ What does each of these show?

I TYPED	IT SHOWED
`cat /dev/time`	
`cat /dev/ticks`	
`ls /dev` (how many names?)	

6. `bind` makes one directory appear in another place. Try it, and undo it:

```
/> ls /mnt
/> bind /rom/songs /mnt
/> ls /mnt
/> unmount /mnt
/> ls /mnt
```

✎ What was in `/mnt` before, during and after? Were any files copied?

7. Type `ns`. It prints the recipe that built your whole tree. ✎ Copy the lines that end in `/bin`. How many directories is `/bin` made of?

Questions

1. ✎ What's a path? What's the difference between `/ram/notes` and `notes`?

2. ✎ What do `.` and `..` mean?

3. ✎ The Hydra has several disks, but only one tree. Which directory is each disk seen at?

4. ✎ `/bin` is a **union**: several directories seen as one. Name two of them. If two of them have a program with the same name, which one runs?

5. ✎ Each task has a **namespace** of its own. You typed `bind` at one prompt. Would a program in another window see `/mnt` change? (Section 9.5 has the experiment.)

Check yourself

☐	I CAN
☐	move around the tree with `cd`, with full paths and with `..`
☐	read a long listing
☐	say which disk a path is on, and how full the disks are
☐	read a device through its file
☐	bind one directory onto another, and take it away again

CHALLENGE: ★ Make a directory `/ram/mybin`, put a script in it, and bind it *after* `/bin` with `bind -a /ram/mybin /bin`. Can you now run the script by its name alone? Use `ns` to see your change, and explain what `-a` did. What would `-b` have done?

LAB 8

Windows

THIS LAB: Manual: Chapter 10, sections 10.1 to 10.4, and 10.8. About 45 minutes. You need a Hydra, on a terminal at least 80 columns wide. The **window key** is CTRL-]: press it, let go, and then press the next key.

Name _____ Date _____

Do

1. Press CTRL-] and then ?. ✎ What appears? How do you make it go away?

2. Press CTRL-] C. ✎ What happened? Which window are you in now? (The bar at the top says.)

3. In the new window, type `echo I am the second window`. Then press each of these, and write what you see.

KEYS	WHAT I SAW
CTRL-] 0	
CTRL-] 1	
CTRL-] W	
CTRL-] N	

4. In window 0, start something slow: `sleep 30`. While it's running, go to the other window and type `ps`. ✎ Is `sleep` still running? What does that tell you about windows and tasks?

5. A window's **group** can have several windows, shown together. In one window, type:

```
/> new-window
/> echo layout columns >/dev/wctl
```

✎ What does the screen look like now? How do you move the keyboard from one to the other?

6. Try the other layouts: `rows`, `grid`, and `tabs` to go back to one at a time. ✎ Which would you use for writing a program in one window and running it in another?

7. Give your window a name, and read it back:

```
/> echo my window >/dev/label
/> cat /dev/label; echo
```

✎ Where on the screen did the name appear?

8. Close the extra windows: type `bye` in each one's shell (or press CTRL-] X). ✎ What happens if you type `bye` in window 0?

Questions

1. ✎ The Hydra can run sixteen programs, but you have one screen. How do windows solve that?

2. ✎ What's the difference between a window and a group?

3. ✎ You're in a full-screen program and need to send it a real CTRL-]. How?

4. ✎ `/dev/label` isn't a file on any disk. What is it? What else in `/dev` belongs to your window?

Check yourself

☐	I CAN
☐	open a new window, and go between windows
☐	list the windows, and close one
☐	show two windows side by side
☐	explain that each window has its own shell, and they all run at once

CHALLENGE: ★ Set up a "workbench": `edit` in one window and a shell in another, side by side. Write a three-line rc script in the first, save it (without leaving the editor), and run it in the second. Change it, save it, run it again. How many keys does one round trip take?

LAB 9

A Command Hunt

THIS LAB: Manual: Chapter 11. About 45 minutes. You need a Hydra, and the manual open at Chapter 11. This lab is a hunt: the manual has the answers, and the Hydra will tell you whether you read it right.

Name _____ Date _____

Do

1. Find the command for each job in Chapter 11, and write its name. (Each is one of the Hydra's programs.)

THE JOB	THE COMMAND
Count the lines, words and bytes in a file	
Show the first few lines of a file	
Show the last few lines of a file	
Find the lines that have a word in them	
Put lines in order	
Show a file a screenful at a time	
Show a file's bytes in hex	
Say whether two files are the same	
Show how full the disks are	
Show which tasks are busiest, over and over	
Stop a program by its *name*	
Do nothing for a number of seconds	

2. Now use them. The file to work on is `/sd/0/sample/as/hi.s`, a program's source. ✎ Write the command you typed, and the answer.

FIND OUT	THE COMMAND I TYPED	THE ANSWER
How many lines it has		
What its first three lines are		

FIND OUT	THE COMMAND I TYPED	THE ANSWER
What its last line is		
How many lines have the word `PRINT` in them		
On which lines the word `window` appears (the line numbers)		

3. Make a small file to play with:

```
/> echo pear >/ram/fruit
/> echo apple >>/ram/fruit
/> echo pear >>/ram/fruit
/> echo fig >>/ram/fruit
```

✎ Write the command, and what it printed.

FIND OUT	THE COMMAND I TYPED	IT PRINTED
The fruit in alphabetical order		
The same, backwards		
The fruit in order, with each kind only once		
How many bytes the file has		
The file's bytes, in hex		

4. ✎ From the hex dump: what's the code for the letter `p`? What byte ends each line?

5. A usage line shows how to type a command. ✎ In the usage line `grep [-chilnsv] [-e] pattern [file ...]`, what do the square brackets mean? What does `...` mean? Which part must you always give?

Questions

1. ✎ `ls` is a program in `/bin`, but `cd` isn't a program at all. What is it, and why can't it be a program? (The end of Chapter 11, and section 8.14.)

2. ✎ Many commands read their **standard input** if you don't name a file. Which of the commands above did you use that way, after a pipe?

3. ✎ What's the difference between `kill` and `slay`?

4. ✎ Which three commands in Chapter 11 do you think you'll use most? Why?

Check yourself

☐	I CAN
☐	find a command in Chapter 11, and read its usage line
☐	use `wc`, `head`, `tail`, `grep`, `sort` and `uniq`
☐	read a hex dump
☐	join two or three commands with pipes to answer a question

CHALLENGE: ★ Which letter do the most programs in `/rom/bin` start with? Work it out with the Hydra, not by counting on your fingers. (Patterns from Lab 5 will help: `ls /rom/bin/s* | wc -l`. Can a `for` loop from Lab 6 try every letter for you? To join a variable to a `*`, rc needs a `^` between them: `$1^*`.)

LAB **10**

Devices Are Files

THIS LAB: Manual: Chapter 12, and sections 13.4, 13.6 and 13.7. About 45 minutes. You need a Hydra at its `/>` prompt. You'll type a few lines of HyForth exactly as they're printed: you don't need to know Forth yet.

Name _____ Date _____

Do

1. Type `ls /dev`. ✎ Pick four names, and say which device you think each belongs to. (The table in section 12.3 will tell you whether you're right.)

2. A control file can be read, to see a device's settings. ✎ What does each of these say?

I TYPED	IT SAID
`cat /dev/serctl`	
`cat /dev/consctl`	
`cat /dev/sd/0/ctl`	
`cat /dev/time`	

3. A control file can be written, to give the device a command. First, a command it doesn't know:

```
/> echo fly >/dev/gpio/ctl
```

✎ What did the Hydra say? Who said it: `echo`, or the driver?

4. Now a command it does know. This one makes pin 2 of the I/O header an output, and switches it on and off:

```
/> echo out 2 >/dev/gpio/ctl
/> echo 1 >/dev/gpio/2
/> cat /dev/gpio/2
/> echo 0 >/dev/gpio/2
/> cat /dev/gpio/2
```

✎ What did the two `cat`s show?

5. The screen is a device too, and it takes its commands *inside* the text sent to it, as **escape sequences** (section 13.7). Type these three lines of HyForth, and then run them:

```
/> : esc 27 emit ;
/> : colours esc ." [31mred " esc ." [32mgreen " esc ." [34mblue " esc ." [0m" cr ;
/> : flag esc ." [1;33;44m Hello, Hydra! " esc ." [0m" cr ;
/> colours
/> flag
```

✎ What did `colours` print, and in which colours? What does `ESC [0m` do? (Try leaving it off the end.)

6. ✎ Using the tables in section 13.7, write the escape sequence that does each job.

THE JOB	THE SEQUENCE
Clear the whole screen	
Move the cursor to row 5, column 10	
Text in bright green	
Text reversed (dark on light)	

Questions

1. ✎ What does a driver do? Why don't programs just talk to the chips themselves?

2. ✎ A control file "works in two directions". What are they?

3. ✎ Every device is a file, so commands that work on files work on devices. Give one example you've used.

4. ✎ What do these error messages mean, when a device gives them: `invalid argument`, `busy`, `no such device`?

5. ✎ Section 5.5 sped up the serial line with `echo b115200 >/dev/serctl`. What else has to change when you do that, and why?

Check yourself

☐	I CAN
☐	find a device's files in `/dev`
☐	read a control file, and write a command to one
☐	tell a driver's error from a typing mistake
☐	change the screen's colours with an escape sequence

CHALLENGE: ★ Write a HyForth word `banner` that clears the screen, moves to the middle, and prints your name in yellow on blue. Then write an rc script that does the same. (In rc, you'll need a real ESC character: section 13.7 and the entry for `echo` in Chapter 11 will help.)

LAB 11

Disks

THIS LAB: Manual: Chapter 14. About 45 minutes. You need a Hydra with an SD card (the emulator has one).

Name _____ Date _____

Do

1. Type `ls /dev/sd`. Each name is a disk. ✎ Fill in the table: read each disk's `ctl` file (`cat /dev/sd/0/ctl`) for the details.

DISK	WHAT KIND IT IS	ITS SIZE	SEEN IN THE TREE AT
0			
x			
r			
s			

2. Try to change the ROM disk:

```
/> echo hello >/rom/test
```

✎ What did the Hydra say? Why can't it be done?

3. How much room does a file really take? Make a tiny file on the SD card, and watch the free space:

```
/> df
/> echo x >/sd/0/tiny
/> ls -l /sd/0/tiny
/> df
/> rm /sd/0/tiny
```

✎ How many bytes long is the file? How much less free space was there after you made it? Why isn't that the same number? (Section 14.5.)

4. Check a disk for faults. `s` is the shared RAM disk:

```
/> fsck s
```

✎ What did it report?

5. `du` adds up the space that files use. ✎ What does each of these say?

I TYPED	IT SAID
`du /sd/0/sample`	
`du /rom/bin`	

6. A disk has a name of its own, its **label**. ✎ What's your SD card's label? (It's in the card's `ctl` file.)

Questions

1. ✎ The Hydra has three kinds of disk. What are they, and what's each one good for?

2. ✎ What's a block? How many bytes is it? What's a cluster?

3. ✎ An SD card works at 3.3 volts, and the Hydra at 5. What sits between them?

4. ✎ Why does the Hydra need its own file system, HydraFS? What must be done to a new card before the Hydra can use it?

5. ✎ Give two ways to move a file from the Hydra to a PC (section 14.7).

Check yourself

☐	I CAN
☐	list the disks, and say what each is

☐	I CAN
☐	find out how much space is free, and how much a directory uses
☐	check a disk for faults
☐	say what happens to each disk's files when the power goes off

CHALLENGE: ★ The storage driver can make another RAM disk while the Hydra is running. Read the `start` and `stop` commands in section 14.4, and work out how. How big a disk can you make? Where does its memory come from? (Watch `free` before and after.) Don't forget to stop it.

LAB 12

Sound

THIS LAB: Manual: Chapter 15, sections 15.1, 15.2, 15.4, 15.6 and 15.7. About an hour. You need a Hydra with headphones or a speaker; in the web emulator, press **Sound** first, and set the speed to **Real time**.

Name _____ Date _____

Do

1. Play one note, by hand. Each line is a command to the sound driver.

```
/> echo patch 0 0 >/dev/sndctl
/> echo note 0 60 >/dev/sndctl
/> echo off 0 >/dev/sndctl
```

✎ What did each of the three lines do? What are the two numbers after `note`?

2. Note 60 is middle C. ✎ Using the table in section 15.2, what are the numbers of these notes?

NOTE	ITS NUMBER
The D above middle C	
The A that an orchestra tunes to	
The C an octave above middle C	
The C an octave below	

3. Change the instrument: `patch 0 40` is a violin. ✎ Try five more patch numbers (section 15.4 lists them), and write down which you tried and what you heard.

4. `play -m` plays music written as text, in **MML**. Try these:

```
/> play -m 0 'o4 l8 c d e f g a b > c'
/> play -m 0 'I40 o5 l4 c c g g a a g2'
/> play -c 0 c e g
```

✎ What does each of these mean in the first line: `o4`, `l8`, `>`? What did the third line play?

5. ✎ Write a tune of your own in MML: at least eight notes, with at least two different lengths and one rest. Play it, change it until you like it, and copy it here.

6. A **score** is a file with a tune for several channels at once. Make this one with `edit /ram/round.mml`:

```
#tempo 120
A I0 o4 l4 c d e c | c d e c | e f g2 | e f g2
B I40 o3 l2 c g | c g | c g | c g
```

Then play it: `play /ram/round.mml`. ✎ What are the `A` and `B` at the start of the lines? What does `#tempo` set?

7. Play the song that comes with HydraOS: `play /rom/songs/scom.zsm`. Press CTRL-C to stop it.

Questions

1. ✎ The YM2151 is an "FM synthesizer" with eight voices. What does "eight voices" let it do?

2. ✎ What's a patch? What's a channel?

3. ✎ Note 69 sounds at 440 Hz. What does "440 Hz" mean? What note would sound at 880 Hz?

4. ✎ A `.mml` file is text that people can read. A `.zsm` file isn't. What's in a ZSM file, and why does the Hydra have both?

5. ✎ In the web emulator, why should the speed be **Real time** when you play music?

Check yourself

☐	I CAN
☐	play a note, and stop it, by writing to `/dev/sndctl`
☐	choose an instrument
☐	write a tune in MML, with lengths, octaves and rests
☐	write a score for two channels, and play it

CHALLENGE: ★ Add a third line to the score for a drum (section 15.6: `x` plays a General MIDI drum, and section 15.4 lists them). Then turn your score into a ZSM file with `play -o`, and play that. Which file is bigger?

LAB 13

Pictures

THIS LAB: Manual: Chapter 16, sections 16.1 to 16.4. About an hour. You need a Hydra with a Vera X. The web emulator has one unless it's been turned off in Setup; for the terminal emulator, start it with `--vera --view`.

Name _____ Date _____

Do

1. Look at the Vera X's screen. ✎ What's on it, before you do anything? How is it different from the serial console?

2. Read the screen's settings: `cat /dev/vid/ctl`. ✎ Copy two of its lines, and say what you think each means.

3. Turn on a **bitmap**, a picture made of dots, under the text, and draw on it. Each line is a command to the screen's `draw` file.

```
/> echo bitmap 320 8 >/dev/vid/ctl
/> echo clear 0 >/dev/vid/draw
/> echo pen 12 >/dev/vid/draw
/> echo circle 160 110 60 >/dev/vid/draw
/> echo pen 10 >/dev/vid/draw
/> echo box 40 30 280 190 >/dev/vid/draw
/> echo pen 11 >/dev/vid/draw
/> echo line 40 30 280 190 >/dev/vid/draw
```

✎ Sketch what's on the screen, and label each shape with the command that drew it.

4. ✎ The bitmap is 320 dots wide and 240 high. Where is the dot (0, 0)? Where is (319, 239)? Which way does *y* go?

5. ✎ Work out, on paper, the commands to draw a house: a square for the walls, a triangle of three lines for the roof, a filled door (`bar`), and a round window (`disc`). Write them here, then type them in. Fix them until it looks like a house.

6. Typing `echo ... >/dev/vid/draw` every time is slow. Put your house's commands in a file, one to a line, without the `echo`, and send the whole file at once:

```
/> edit /ram/house.txt
/> cat /ram/house.txt >/dev/vid/draw
```

✎ Why does that work?

7. Write your name on the picture with `text`, and then turn the bitmap off with `echo bitmap off >/dev/vid/ctl`.

Questions

1. ✎ What does the Vera X add to a Hydra? Name four things.

2. ✎ What's a pixel (a dot)? How many are there in a 320 by 240 bitmap?

3. ✎ `bitmap 320 8` means 320 dots across, and 8 bits for each dot. How many colours can 8 bits choose between? How many could 4 bits?

4. ✎ What's the difference between `box` and `bar`? Between `circle` and `disc`?

5. ✎ The picture is drawn by writing text to a file. What does that make easy that a special drawing program wouldn't?

Check yourself

☐	I CAN
☐	turn a bitmap on and off
☐	find a point from its *x* and *y*
☐	draw lines, boxes and circles, in the colours I choose
☐	keep a drawing as a file of commands, and draw it again

CHALLENGE: ★ Write an rc script that draws ten circles, each bigger than the last, each in a different colour. You'll need a `for` loop, and `calc` to work out each radius. Then make them all share a centre… or not.

LAB 14

Pins, and the System's Own Files

THIS LAB: Manual: Chapter 17, sections 17.1, 17.2, 17.5, 17.7 and 17.8. About an hour. Part A needs a real Hydra, an LED, a 2.2K resistor and three wires; on an emulator, do its steps 1 and 2 only. Part B works anywhere.

Name _____ Date _____

Do: Part A, a light on a pin

1. ⚠ Read the caution in section 17.1 first. Then make pin PA2 an output, and switch it on and off:

```
/> echo out 2 >/dev/gpio/ctl
/> echo 1 >/dev/gpio/2
/> cat /dev/gpio/2
/> echo 0 >/dev/gpio/2
/> cat /dev/gpio/ctl
```

✎ What does the `ctl` file tell you about the eight pins?

2. ✎ On the header J27, which pin number is PA2? Which is +5 V, and which is ground? (The table in section 17.1.)

3. With the power **off**, wire the LED as section 17.1 says: from +5 V, through the 2.2K resistor, through the LED, to PA2. ✎ Draw your circuit. Which leg of the LED goes towards +5 V?

4. Switch on, make PA2 an output again, and write 0 and then 1 to it. ✎ Which one lights the LED? Why is it "the wrong way round"?

5. Make it blink. In rc (type `rc` first):

```
% while(~ 1 1) { echo 0 >/dev/gpio/2; sleep 1; echo 1 >/dev/gpio/2; sleep 1 }
```

(`~ 1 1` is a test that's always true, so the loop goes round for ever.) ✎ How do you stop it? How would you make it blink faster?

Do: Part B, the system as files

6. The kernel shows the tasks as files. Type `ls /proc`, and then look at your own shell (use the task number that `ps` shows for `forth`):

```
/> ps
/> cat /proc/1/status
/> cat /proc/1/fd
```

✎ What does `status` tell you? What are the files your shell has open (`fd`)?

7. The environment is a directory of variables. ✎ What's in it? What does `cat /env/prompt` show?

8. Set the clock. (Use today's date and the time now, in exactly this form.)

```
/> echo 2026-10-12 09:41:00 >/dev/time
/> date
/> cat /dev/rtc
```

✎ What did `/dev/rtc` say? Will your Hydra still know the date tomorrow?

Questions

1. ✎ A pin can be an input or an output. What's the difference? Give one thing you could connect to each.

2. ✎ Why does the LED need a resistor?

3. ✎ PA0 and PA1 have a second job. What is it, and what does that mean for using them as ordinary pins?

4. ✎ `/proc` isn't on any disk. Where do its "files" come from? What reads them, besides you? (Section 29.11 has a surprise.)

5. ✎ You've now seen pins, the clock, tasks and variables, all as files. What's the advantage of making everything a file?

Check yourself

☐	I CAN
☐	make a pin an output, and set it; or an input, and read it
☐	wire an LED to a pin safely
☐	look at a task through `/proc`
☐	read and set the clock

CHALLENGE: ★ Wire a push button from PA3 to ground. An input with nothing connected reads 1 (section 17.2), so PA3 reads 1 until the button is pressed, and 0 while it's held. Write a loop that lights the LED only while the button is down. (No real Hydra? Then write a script that prints the name of every task, by reading `/proc`.)

LAB 15

Numbers and Bases

THIS LAB: Manual: Chapter 18, sections 18.1 to 18.4, and 18.7. About 45 minutes. You need a Hydra. `calc` does the sums: put each sum in single quotes, so the shell leaves its signs alone.

Name _____ Date _____

Do

1. First, guess. Then ask the Hydra, and write what it really says.

THE SUM	MY GUESS	THE HYDRA SAYS
`calc '2^10'`		
`calc '2^64'`		
`calc '10 / 4'`		
`calc '1/3 + 1/6'`		
`calc '0.1 + 0.2'`		
`calc '1/3 * 3'`		
`calc 'sqrt 16'`		
`calc 'sqrt 2'`		
`calc 'sqrt(-4)'`		

2. ✎ Most calculators say that 10 ÷ 4 is 2.5, and many computers say that 0.1 + 0.2 is 0.30000000000000004. What does the Hydra do differently?

3. `calc -d` sets how many digits the functions work to. ✎ Write the square root of 2 to 30 digits.

4. `calc -b` writes the answer in another base. Fill in the table.

DECIMAL	IN BINARY (`-b b`)	IN HEX (`-b x`)	IN OCTAL (`-b o`)
10			
100			
255			
1000			

5. A number can *say* its own base, with `#` and a letter. ✎ What do these come to?

THE SUM	THE ANSWER
`calc '#b1010'`	
`calc '#xFF'`	
`calc '#b1010 + #x0F'`	
`calc -b x '#xFF + 1'`	

6. Bases don't stop at 16. ✎ Try `calc -b t 10` and `calc -b z 1000000`. What are these two bases, and what digits does each use? (Section 18.4.)

Questions

1. ✎ The Hydra has four kinds of number. Name them, and give an example of each.

2. ✎ Why does 1/3 × 3 come to exactly 1 on the Hydra, when on many calculators it comes to 0.9999999?

3. ✎ In base 2 there are two digits, and in base 16 sixteen. How many digits does base 8 need? What's the biggest digit in base 8?

4. ✎ Programmers like hex because it lines up with binary. Show how: write `$A5` in binary, a hex digit at a time.

5. ✎ What does `calc 1/0` say? What should a program do when that can happen?

Check yourself

☐	I CAN
☐	tell a whole number, a fraction, a decimal and a complex number apart
☐	get an exact answer, or a decimal one, as I choose
☐	write a number in binary, octal and hex, and read one back
☐	set the precision of a function like `sqrt`

CHALLENGE: ★ How many digits has 2 to the power 1000? (`calc '2^1000'` will show you the number. Counting them by eye isn't the best way: `wc -c` counts characters, the line's end included.) Then read about **balanced ternary** in section 18.4, where the digits are minus one, zero and plus one. Write 5, 6 and 7 in it by hand, and check with `calc -b c`.

LAB 16

HyForth: The Stack

THIS LAB: Manual: Chapter 19, sections 19.1 to 19.6. About an hour. You need a Hydra at its `/>` prompt: that prompt *is* HyForth.

Name _____ Date _____

Do

1. Be the computer. For each line, write what you think the stack holds afterwards (the top on the right), then type the line, and then `.s` to see. Clear the stack between lines with as many `drop`s as it takes.

THE LINE	I THINK THE STACK IS	.s SAYS
`1 2 3`		
`1 2 3 swap`		
`1 2 3 rot`		
`1 2 over`		
`5 dup *`		
`10 2 3 + *`		
`10 2 * 3 +`		

2. Turn these sums into Forth, and get the Hydra to print the answers. ✎ Write the line you typed.

THE SUM	IN FORTH	THE ANSWER
4 + 5		
(4 + 5) × 2		
100 – (7 × 8)		
(12 + 4) ÷ (9 – 5)		
The remainder when 100 is divided by 7		

3. Define a word, and use it:

```
/> : square ( n -- n*n ) dup * ;
/> 9 square .
/> see square
```

✎ What does `( n -- n*n )` mean? Does HyForth do anything with it?

4. ✎ Now write your own. Test each one with at least two numbers, and write the definition here.

THE WORD	WHAT IT DOES	MY DEFINITION
`double`	(*n -- 2n*)	
`cube`	(*n -- n³*)	
`average`	(*a b -- the number half way between*)	
`hello`	(--) prints a greeting	

5. Start a definition, and don't finish it: type `: oops 1 2` and press RETURN. ✎ What happened to the prompt? How do you get out?

6. Type `drop` when the stack is empty. Then type `1 0 /`. ✎ What does HyForth say each time?

Questions

1. ✎ What is the stack? Why is it called that?

2. ✎ In Forth, `2 3 +` adds, where school writes 2 + 3. What's the advantage of putting the `+` last?

3. ✎ What's the difference between `.` and `.s`?

4. ✎ What goes wrong with `: double 2*;`? (Look closely.)

5. ✎ A number on Forth's stack is a 16-bit cell. What's the biggest number it holds? What does `32767 1 +` `.` print, and why?

Check yourself

☐	I CAN
☐	say what the stack holds after a line of Forth
☐	turn a sum into Forth
☐	use `dup`, `drop`, `swap`, `over` and `rot`
☐	define a word of my own, with a stack diagram
☐	get out of an unfinished definition

CHALLENGE: ★ Write `f>c` (*fahrenheit -- celsius*), which turns a temperature in Fahrenheit into Celsius: take away 32, multiply by 5, divide by 9. Check it with 212 (100) and 32 (0). Then write `c>f`. What happens with 98? Why isn't the answer exact, and how could you get one more digit out of it?

LAB 17

HyForth: Decisions, Loops and Programs

THIS LAB: Manual: Chapter 19, sections 19.7 to 19.17. About an hour. You need a Hydra, and Lab 16 done.

Name _____ Date _____

Do

1. A comparison leaves a **flag**. ✎ What does each print?

THE LINE	IT PRINTS	SO THE FLAG FOR "TRUE" IS
`5 3 > .`		
`5 3 < .`		
`7 7 = .`		
`0 0= .`		

2. `if` uses a flag. Type this, and try it with 5, with -5 and with 0:

```
/> : sign ( n -- ) dup 0< if ." negative" else ." not negative" then drop cr ;
```

✎ Why is the `dup` there? What would happen without the `drop`?

3. ✎ Write `big?` (*n* --), which prints "big" for a number over 100, and "small" otherwise.

4. A counted loop: `do` ... `loop`. ✎ What does each of these print? Guess first.

THE DEFINITION, AND THE LINE	I GUESS	IT PRINTS
`: a 5 0 do i . loop ;` then `a`		
`: b 11 1 do i . loop ;` then `b`		

THE DEFINITION, AND THE LINE	I GUESS	IT PRINTS
`: c 10 0 do i . 2 +loop ;` then `c`		
`: d 3 0 do ." Hip! " loop ." Hooray!" ;` then `d`		

5. ✎ Write `stars` (*n* --), which prints that many `*` on one line. (`[char] * emit` prints one.) Then use it to write `triangle` (*n* --), which prints a triangle *n* rows high.

6. A variable keeps a number between words:

```
/> variable score
/> 0 score !
/> : hit ( -- ) 10 score +! ;
/> hit hit hit
/> score @ .
```

✎ What do `!`, `@` and `+!` do? What's on the stack after you type just `score`?

7. Put your words from this lab in a file, `/sd/0/work/lab17.fs`, with `edit`. Then leave them behind: type `bye` in window 0 (a new shell starts), and bring them back with `include /sd/0/work/lab17.fs`. ✎ Did they come back?

Questions

1. ✎ `if` ... `then`: in English, "then" comes before what you do. What does `then` mean in Forth?

2. ✎ `10 0 do` counts from 0 to 9. Why not to 10? What would `0 0 do` do, and what's the cure?

3. ✎ Typed at the prompt, `5 0 do i . loop` gives `do: compile only`. What does that mean?

4. ✎ What's the difference between a `variable`, a `constant` and a `value`?

5. ✎ FizzBuzz (section 19.17) is built from three small words and one loop. Why is that better than one long definition?

Check yourself

☐	I CAN
☐	compare numbers, and use the flag in an `if`
☐	write a `do` loop, and use `i`
☐	keep a value in a variable, and change it
☐	keep my words in a file, and `include` it
☐	test a small word by itself before using it in a bigger one

CHALLENGE: ★ Write `primes` (*n* --), which prints the prime numbers up to *n*. Build it from small words: first `divides?` (*a b* -- *flag*), true if *b* goes into *a*; then `prime?` (*n* -- *flag*), which tries every number from 2 up; then the loop. How far can it go before it gets slow?

LAB 18

hylang: Expressions and Functions

THIS LAB: Manual: Chapter 21, sections 21.1 to 21.5. About an hour. You need a Hydra. Type `hylang` to start it, and `(exit)` to leave.

Name _____ Date _____

Do

1. First, guess. Then ask hylang.

THE EXPRESSION	MY GUESS	HYLANG SAYS
`(+ 1 2 3 4)`		
`(* 2 (+ 3 4))`		
`(- 10 (* 2 3))`		
`(/ 10 4)`		
`(> 5 3)`		
`(> 3 5)`		
`(+ "Hydra" "-" 16)`		

2. Turn these sums into hylang. ✎ Write the expression, and its value.

THE SUM	IN HYLANG	ITS VALUE
(4 + 5) × 2		
100 – (7 × 8)		
(12 + 4) ÷ (9 – 5)		
1/2 + 1/3 + 1/6		

3. Give a value a name, and use it:

```
hylang> (def {age} 12)
hylang> (* age 365)
hylang> (def {name} "Sam")
hylang> (print "Hello," name)
```

✎ Why is the name in braces in `def`, but not when you use it?

4. Make a function, and use it:

```
hylang> (fun {square n} {* n n})
hylang> (square 9)
hylang> (square (square 3))
```

✎ In `(fun {square n} {* n n})`, which part is the function's name? Its formal? Its body?

5. ✎ Now write your own. Test each with at least two values, and write the definition here.

THE FUNCTION	WHAT IT GIVES	MY DEFINITION
`(double n)`	Twice *n*	
`(cube n)`	$n \times n \times n$	
`(average a b)`	The number half way between	
`(greet who)`	Prints "Hello, " and the name	

6. Make these mistakes on purpose. ✎ Write what hylang says.

I TYPED	HYLANG SAID
`(squre 5)`	
`(square 1 2)`	
`(square)`	
`(+ 1 "a")`	

7. Type `(+ 1 2` and press RETURN, without the last parenthesis. ✎ What happens? How do you finish?

Questions

1. ✎ In hylang, the function comes first: `(+ 2 3)`. What can `+` do in hylang that it can't do in school arithmetic, because of that?

2. ✎ What do `T` and `NIL` mean?

3. ✎ What's the difference between `{* n n}` and `(* n n)`? When is each one worked out?

4. ✎ `(/ 10 4)` gives `5/2`. How would you get 2.5? How would you get just 2? (Appendix B.4.)

5. ✎ `(square)`, with no argument, gave you a function back. What might that be useful for?

Check yourself

☐	I CAN
☐	write a nested expression, and say what it's worth
☐	name a value with `def`
☐	define a function with `fun`, and call it
☐	read hylang's commonest error messages
☐	balance my parentheses

CHALLENGE: ★ Write `(celsius f)`, which turns Fahrenheit into Celsius: take away 32, multiply by 5, divide by 9. What does `(celsius 98)` give? Compare that with what HyForth gave you in Lab 16. Then write `(hypotenuse a b)`, using `sqrt`. What's `(hypotenuse 3 4)`? And `(hypotenuse 1 1)`?

LAB 19

hylang: Lists, Decisions and Programs

THIS LAB: Manual: Chapter 21, sections 21.6 to 21.16. About an hour. You need a Hydra, and Lab 18 done.

Name _____ Date _____

Do

1. A list is values in braces. ✎ What does each of these give?

THE EXPRESSION	ITS VALUE
`(def {l} {5 3 8 1})`	
`(len l)`	
`(fst l)`	
`(tail l)`	
`(sort l)`	
`(reverse l)`	
`(sum l)`	
`(range 5)`	
`(range 1 11)`	

2. `map` does something to every item, and `filter` keeps some of them. ✎ What does each give?

THE EXPRESSION	ITS VALUE
`(map (fn {n} {* n 2}) {1 2 3})`	
`(filter (fn {n} {> n 2}) {1 2 3 4})`	
`(sum (map (fn {n} {* n n}) (range 1 11)))`	

3. ✎ In one expression each: the sum of the numbers from 1 to 100; and the even numbers from 2 to 20, as a list.

4. A decision: `if` takes a test, then a value for true and a value for false.

```
hylang> (fun {size n} {if (> n 100) "big" "small"})
hylang> (size 5)
hylang> (size 500)
hylang> (map size {5 500 50 5000})
```

✎ What did the last line give?

5. A loop, and a function that calls itself. ✎ What does each print or give?

THE EXPRESSION	WHAT IT DOES
`(dotimes {i 3} (print "Hip!"))`	
`(each {x {1 2 3}} (print (* x x)))`	
`(fun {fact n} {if (< n 2) 1 (* n (fact (- n 1))))})`, then `(fact 5)`	
`(fact 30)`	

6. A hash keeps values under names:

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
hylang> (pet :name)
hylang> (hash-put pet {:colour "brown"})
hylang> (hash-keys pet)
```

✎ What did the last line give? What kind of thing is `:name`?

7. Put a program in a file, `/sd/0/work/lab19.hl`, with `edit`: two functions from this lab, and a line that prints something with them. Run it from the shell with `hylang /sd/0/work/lab19.hl`. ✎ Write your program here.

Questions

1. ✎ `{+ 1 2}` and `(+ 1 2)` look alike. What's the difference? What does `(eval {+ 1 2})` do?

2. ✎ HyForth can't work out `(fact 30)`. Why can hylang?

3. ✎ `fact` calls itself. Why doesn't it go on for ever?

4. ✎ What does `map` save you from writing?

5. ✎ In FizzBuzz (section 21.16), the function `fizz` *gives* a value, and doesn't print anything. Why is that the better way?

Check yourself

☐	I CAN
☐	make a list, and take it apart
☐	use `map` and `filter` with a function of my own
☐	choose with `if` and `cond`
☐	write a function that calls itself, and say why it stops
☐	keep a program in a file, and run it

CHALLENGE: ★ Write `(primes n)`, which gives a list of the prime numbers up to *n*. One way: write `(prime? n)` first, using `any?` or `filter` over `(range 2 n)`, and then `filter` the numbers from 2 to *n* with it. How long does `(primes 100)` take? Compare with your HyForth version from Lab 17.

LAB 20

HyBASIC: First Programs

THIS LAB: Manual: Chapter 23, sections 23.1 to 23.8. About an hour. You need a Hydra. Type `basic` to start HyBASIC, and `SYSTEM` to leave.

Name _____ Date _____

Do

1. HyBASIC as a calculator. First, guess. Then ask.

I TYPED	MY GUESS	HYBASIC SAID
`PRINT 2 + 3 * 4`		
`PRINT (2 + 3) * 4`		
`PRINT 10 / 4`		
`PRINT 10 \ 4`		
`PRINT 10 MOD 4`		
`PRINT 2 ^ 10`		
`PRINT "Hy" + "dra"`		

2. Variables. Type these, one at a time:

```
> AGE = 12
> NAME$ = "Sam"
> PRINT NAME$; " is"; AGE
> PRINT NAME$, AGE
> AGE = AGE + 1
> PRINT AGE
```

✎ What's the difference between `;` and `,` in a `PRINT`? What does `AGE = AGE + 1` mean? (It isn't an equation!)

3. A program. Type the lines, then `LIST`, then `RUN`.

```
> 10 FOR I = 1 TO 5
> 20 PRINT I; "times 7 is"; I * 7
> 30 NEXT I
```

Now change it without typing it all again: make it go up to 12, and add a line 5 that prints a title. ✎ Write what you typed to make each change.

4. ✎ Add a line 40 that prints "Done." Then put a line *between* 20 and 30 that prints a row of dashes. What number did you give it? Why are lines usually numbered in tens?

5. Keep the program, clear it, and bring it back:

```
> SAVE "/sd/0/work/seven.bas"
> NEW
> LIST
> LOAD "/sd/0/work/seven.bas"
> LIST
```

✎ What did the first `LIST` show? And the second?

6. A program that asks. Type `NEW`, and then:

```
> 10 INPUT "What's your name"; N$
> 20 INPUT "How old are you"; A
> 30 IF A < 13 THEN PRINT "Hello, young "; N$ ELSE PRINT "Hello, "; N$
> 40 PRINT "Next year you'll be"; A + 1
```

Run it twice, with different answers. ✎ What happens if you answer the second question with a word?

7. ✎ Change line 30 into a block `IF` … `ELSEIF` … `ELSE` … `END IF` (section 23.7) with three cases: under 13, 13 to 19, and over 19. Write your lines here.

Questions

1. ✎ What's the difference between a line typed with a number in front and one without?

2. ✎ `NAME$` and `AGE` are both variables. What does the `$` mean?

3. ✎ A variable you've never given a value isn't an error. What is it? Why can that hide a mistake?

4. ✎ Which should come first in a program that works out an average: `INPUT`, the sum, or `PRINT`? What does that tell you about how BASIC reads a program?

5. ✎ What do `NEW`, `LIST`, `RUN`, `SAVE` and `LOAD` each do?

Check yourself

☐	I CAN
☐	use HyBASIC as a calculator
☐	keep numbers and strings in variables
☐	type a program in, list it, run it, and change a line
☐	save a program, and load it again
☐	ask a question with `INPUT`, and decide with `IF`

CHALLENGE: ★ Write a program that asks for a number, and prints its times table from 1 to 12, lined up neatly with `PRINT USING` (section 23.14). Then make it ask "Another?" at the end, and go round again if the answer is yes.

LAB **21**

HyBASIC: Bigger Programs

THIS LAB: Manual: Chapter 23, sections 23.8 to 23.14. About an hour. You need a Hydra, and Lab 20 done. The programs in this lab have no line numbers: write each in a file with `edit`, and run it from the shell with `basic` and its name.

Name _____ Date _____

Do

1. A loop that doesn't know how long it'll go on. Make `/sd/0/work/double.bas`:

```
' double.bas - how many doublings to a million?
N = 1
COUNT = 0
DO WHILE N < 1000000
  N = N * 2
  COUNT = COUNT + 1
LOOP
PRINT COUNT; "doublings make"; N
```

Run it with `basic /sd/0/work/double.bas`. ✎ What did it print? Why is a `DO` loop better here than a `FOR`?

2. Strings. Start `basic`, and try these. ✎ What does each print?

I TYPED	IT PRINTED
`A$ = "Hydra-16"`	(nothing)
`PRINT LEN(A$)`	
`PRINT LEFT$(A$, 5)`	
`PRINT RIGHT$(A$, 2)`	
`PRINT MID$(A$, 2, 3)`	
`PRINT UCASE$(A$)`	
`PRINT INSTR(A$, "-")`	

3. ✎ Write a program that asks for a word, and prints it backwards. (You'll need a `FOR` loop that counts *down*, with `STEP -1`, and `MID$`.)

4. An array, and data to fill it. Make `/sd/0/work/marks.bas`:

```
' marks.bas - the average of some marks
DIM MARK(5)
TOTAL = 0
FOR I = 1 TO 5
  READ MARK(I)
  TOTAL = TOTAL + MARK(I)
NEXT I
PRINT "The average is"; TOTAL / 5
DATA 7, 9, 4, 8, 10
```

✎ What does it print? Is that what you'd expect from a calculator? Change it to print a decimal instead (`FIXED`, in Appendix C).

5. ✎ Add to `marks.bas`: after the average, print the highest mark, and the lowest. Write the lines you added.

6. A procedure of your own. Make `/sd/0/work/box.bas`:

```
' box.bas - boxes of stars
BOX 3, 5
BOX 2, 10
END

SUB BOX (ROWS, COLS)
  FOR R = 1 TO ROWS
    PRINT STRING$(COLS, "*")
  NEXT R
  PRINT
END SUB
```

✎ What does it print? What are `ROWS` and `COLS`, inside the `SUB`?

7. ✎ Add a `FUNCTION AREA(ROWS, COLS)` to `box.bas`, and make the program print each box's area under it.

Questions

1. ✎ `FOR`, `DO WHILE`, `DO` … `LOOP UNTIL`: when would you choose each?

2. ✎ `DIM MARK(5)` makes how many places? (It isn't five.)

3. ✎ What's the difference between a `SUB` and a `FUNCTION`?

4. ✎ A variable inside a `SUB` is the `SUB`'s own. Why is that a good thing in a big program?

5. ✎ Why do the programs in this lab have their lines moved in by two spaces inside loops and `SUB`s? Does HyBASIC care?

Check yourself

☐	I CAN
☐	choose the right kind of loop
☐	take a string apart, and put strings together
☐	keep a list of values in an array, and fill it from `DATA`
☐	write a `SUB` and a `FUNCTION`, and pass them values
☐	write a program in a file, without line numbers, and run it

CHALLENGE: ★ Write a quiz. Keep five questions and their answers in `DATA` lines. Ask each question, check the answer (make `Paris`, `paris` and `PARIS` all count: `UCASE$`), keep score, and at the end say how many were right, and something kind or cheeky depending on the score.

LAB 22

Your Language Meets the Hydra

THIS LAB: Manual: the second chapter of your language: Chapter 20 (HyForth), Chapter 22 (hylang) or Chapter 24 (HyBASIC). About an hour. You need a Hydra, with sound; the last step needs a Vera X. Choose **one** language for the whole lab.

Name _____ Date _____ My language _____

Each step is a job to do *from your language*. The manual's chapter has what you need; your job is to find it, try it, and write down the line (or lines) that worked.

Do

1. **Run a command.** Make your language run the shell command `date`, and show what it printed.

2. **Write a file.** Write three lines of text into `/ram/log.txt`. Leave your language, and check with `cat`.

3. **Read it back.** Read `/ram/log.txt` from your language, and print each line with its line number in front.

4. **The screen.** Clear the screen, and print your name near the middle of it, in a colour.

5. **Sound.** Play three notes, one after another: C, E and G.

6. **Time.** Print "Ready...", wait two seconds, and print "Go!".

7. **A picture** (with a Vera X). Turn on a bitmap, and draw a box with a circle inside it.

8. ✎ Put three of these together into one small program, in a file in `/sd/0/work`. Say what it does.

Questions

1. ✎ Which job was easiest in your language? Which was hardest? Why, do you think?

2. ✎ Did your language need a **library** for any of these? Which, and how did you load it?

3. ✎ In Lab 12 you played notes by writing to `/dev/sndctl`, and in Lab 13 you drew by writing to `/dev/vid/draw`. Could your language have done these jobs *that* way, with nothing but its file words? What does that tell you about how the libraries work?

4. ✎ What happens when your program asks for something that isn't there (a file that doesn't exist, or a picture with no Vera X)? How can the program find out, and carry on?

Check yourself

☐	IN MY LANGUAGE, I CAN
☐	run another program, and use what it prints
☐	write a file, and read one
☐	control the screen: clear it, move the cursor, change colour
☐	make a sound
☐	wait

CHALLENGE: ★ Do steps 1, 2 and 5 again in one of the *other* two languages. Then fill in a table of your own: for each job, the line in each language. Which language's line would you rather explain to a friend?

LAB 23

Three Languages

THIS LAB: Manual: Chapter 25. About an hour. You need a Hydra, and the Learning chapter of at least one language behind you.

Name _____ Date _____

Do

1. Type in the guessing game in your language (section 25.2, 25.3 or 25.4), as a file in `/sd/0/work`, and run it. Play five games. ✎ Write how many guesses each took.

GAME	1	2	3	4	5
Guesses					

2. ✎ What's the best way to play? With that way, what's the *most* guesses you could ever need for a number from 1 to 100? Why?

3. Change the game: give the player only seven guesses. If they run out, say what the number was. ✎ Write the lines you changed or added.

4. Now read the game in the two languages you *didn't* type. For each, find the line that does each job, and copy it.

THE JOB	IN (LANGUAGE)	IN (LANGUAGE)
Chooses the secret number		
Reads your guess		

THE JOB	IN (LANGUAGE)	IN (LANGUAGE)
Says "Too small."		
Goes round again		
Counts the guesses		

5. The race (section 25.6). Run the counting race in all three languages on *your* Hydra. ✎ Write the times.

LANGUAGE	TIME TO COUNT TO 10,000	TIMES SLOWER THAN THE FASTEST
HyForth		
hylang		
HyBASIC		

6. ✎ Change each race to count to 40,000 instead. Predict the times first. Were you right? (One of the three will surprise you with the number it prints. Which, and why? Section 19.3's note has the reason.)

Questions

1. ✎ hylang's game has no loop. How does it go round again?

2. ✎ HyForth's `ask` is the longest of the three ways to read a number. What do HyBASIC and hylang do for you that HyForth makes you do yourself?

3. ✎ Why is HyForth so much quicker in the race? What do you give up for that speed?

4. ✎ "The time that matters is yours." What does the manual mean by that?

5. ✎ Which language would you choose for each of these, and why: a quiz for your class; working out the 200th Fibonacci number; flashing a light a thousand times a second?

Check yourself

☐	I CAN
☐	read a small program in a language I haven't learned, and say roughly what each line does
☐	time a piece of a program
☐	give a reason for choosing one language over another

CHALLENGE: ★ Turn the game round: *you* think of a number, and the Hydra guesses. After each guess you type `s` (too small), `b` (too big) or `r` (right). The Hydra should never need more than seven guesses. Write it in your language.

LAB 24

The 65C02, on Paper

THIS LAB: Manual: Chapter 26. About an hour. You need a pencil, the manual, and Appendix D. No Hydra: in this lab, *you* are the processor.

Name _____ Date _____

Do

1. ✎ Fill in the table of the 65C02's registers (section 26.3).

REGISTER	HOW MANY BITS	WHAT IT'S FOR
A		
X		
Y		
S		
PC		
P		

2. Hex arithmetic, as the processor does it: in one byte, with a carry. ✎ Work these out.

THE SUM	THE BYTE LEFT IN A	THE CARRY (0 OR 1)
`$10 + $20`		
`$0F + $01`		
`$80 + $80`		
`$FF + $01`		
`$F0 + $20`		

3. Run this program in your head. After each instruction, write what A and X hold, and the Z flag (1 if the instruction's result was zero).

THE INSTRUCTION	A	X	Z	WHAT IT DID
`lda #$03`				

THE INSTRUCTION	A	X	Z	WHAT IT DID
`tax`				
`clc`				
`adc #$04`				
`dex`				
`dex`				
`dex`				

4. This one has a loop. Run it until it reaches `rts`. ✎ How many times does it go round? What's in A at the end?

```
        lda #$00
        ldx #$04
loop:   clc
        adc #$05
        dex
        bne loop
        rts
```

5. ✎ What does that program work out? Change it so that it works out 7 × 6.

6. Each instruction has an **addressing mode**: its way of saying which byte it means (section 26.6). ✎ Name the mode of each, and say where the byte comes from.

THE INSTRUCTION	ITS ADDRESSING MODE	THE BYTE IT LOADS IS
`lda #$41`		
`lda $22`		
`lda $1234`		
`lda $1234,x`		
`lda ($22),y`		

7. Be the assembler. Using Appendix D, turn these three instructions into bytes, in hex.

THE INSTRUCTION	ITS BYTES
`lda #$41`	
`sta $22`	
`rts`	

Questions

1. ✎ What's the difference between machine code and assembly language?

2. ✎ What does the program counter do? What changes it, besides just going on to the next instruction?

3. ✎ `lda #$41` and `lda $41` differ by one character. What's the difference in what they do?

4. ✎ A branch like `bne` can only reach instructions close by. How far? What do you use to go further?

5. ✎ What's the zero page, and why do programs like to keep their variables there?

6. ✎ The 65C02 can't multiply. How did the program in step 4 manage?

Check yourself

☐	I CAN
☐	name the registers, and say what each is for
☐	add two bytes in hex, with the carry
☐	follow a short program by hand, flags and all
☐	tell the addressing modes apart
☐	look up an instruction's opcode in Appendix D

CHALLENGE: ★ Write, on paper, a program that leaves in A the sum 1 + 2 + 3 + … + 10. Use X as the counter. Run it by hand to check it. What's the answer in hex? What would go wrong if you added up to 30 instead, and how could the program find out?

LAB 25

Your First Machine Code

THIS LAB: Manual: Chapter 27, sections 27.1 to 27.9, and 27.18. About an hour. You need a Hydra, and Lab 24 done. Work in `/ram` while you experiment, and copy what you want to keep to `/sd/0/work`.

Name _____ Date _____

Do

1. Type in `stars.s` from section 27.1, with `edit`. Assemble it, and run it:

```
/ram> as stars.s
/ram> ./stars
```

✎ What did `as` print? What does that mean?

2. ✎ Find each of these in your `stars.s`, and copy the line.

FIND	THE LINE
A comment	
A constant being made	
A variable being made	
A label	
A system call	
The instruction that ends the program	

3. Change `ROWS` to 8, assemble, and run. Then change the `'*'` to another character. ✎ How many lines did you have to change each time? Why is `ROWS` a constant, and not just a 5 in the code?

4. Break it on purpose, one mistake at a time, and see what `as` says. (Put each one right before the next.)

THE MISTAKE	WHAT AS SAID
Spell `lda` as `lad`	

THE MISTAKE	WHAT AS SAID
Take the `#` off `lda #1`	(It assembles! What does the program do now, and why?)
Jump to a label that isn't there: `jsr lime`	
Leave out `.include "hydra.inc"`	

5. Ask for a labels file, and look at it:

```
/ram> as -l stars.s
/ram> cat stars.lbl
```

✎ At what address does `main` start? And `line`? Why isn't `main` at `$0800`, where a program starts?

6. ✎ How big is the program `stars`, in bytes (`ls -l`)? How big is its source, `stars.s`? What's in the source that isn't in the program?

7. Make it your own: change `stars.s` so that the triangle points the other way, with the longest row first. ✎ Write the lines you changed.

Questions

1. ✎ What does an assembler do? What goes in, and what comes out?

2. ✎ Why does `line` keep its count in the variable `count`, and not in the X register?

3. ✎ `row` and `count` are on the zero page. How does the source say so? Where exactly do they end up? (Your labels file knows.)

4. ✎ What's a system call? Which one does `stars` use, and what does it need in which register?

5. ✎ `lda #1` and `lda 1` both assemble. Why is the second one almost always a bug?

Check yourself

☐	I CAN
☐	type in a source file, assemble it and run it
☐	read `as`'s error messages, and fix what they point at
☐	make and read a labels file
☐	use constants, variables, labels and a subroutine
☐	call HydraOS to print a character

CHALLENGE: ★ Write `count.s`, which prints the digits 0 to 9 on one line. (A digit's character code is `$30` plus the digit.) Then make it print 00 to 99, ten to a line. You'll want a subroutine that prints one digit, and two counters.

LAB 26

Taking Programs Apart

THIS LAB: Manual: Chapter 28, sections 28.1 to 28.3; and Chapter 29, sections 29.1 to 29.7. About an hour. You need a Hydra, and `stars.s` from Lab 25, in `/ram`.

Name _____ Date _____

Do: the disassembler

1. Assemble `stars.s` with a labels file, and take the program apart again:

```
/ram> as -l stars.s
/ram> dis stars
/ram> dis -l stars.lbl stars
```

✎ What's the difference between the two listings? What has `dis` no way of getting back from the program?

2. Put it back together, and compare:

```
/ram> dis stars >stars2.s
/ram> as stars2.s stars2
/ram> cmp stars stars2 && echo the same
```

✎ What did `cmp` say? What does that prove?

3. Now look inside a program you didn't write: `dis /rom/bin/echo | more`. ✎ Find one system call that `echo` makes. Which is it?

Do: the debugger

4. Start `stars` in the debugger, and load the names:

```
/ram> db ./stars
db> l /lib/as/hydra.inc
db> l stars.lbl
db> d main 9
```

✎ Copy the first line `db` printed when it started. What's in PC, and in A?

5. Step through the program an instruction at a time. After each `s`, write what changed.

I TYPED	PC	A	THE NEXT INSTRUCTION	WHAT JUST HAPPENED
`s`				
`s`				
`s`				
`s`				
`s`				

6. Look at the program's variables: `m row 2`. ✎ What's in `row`? What's in `count`, and why?

7. Let it run to a **breakpoint**. Set one where the program is about to print a new line, and continue:

```
db> b line+B
db> c
db> r
db> c
db> c
```

✎ What does the program print between each stop? What does `r` show, each time?

8. Change the program while it's stopped. Find the address of the `lda #$2A` (the star) with `d line 3`, and write a different character's code over the `2A`. For example, if the `A9 2A` is at `$0844`:

```
db> w 0845 23
db> x
db> c
```

✎ What does the program print now? Is `stars.s` changed? Is the file `stars`?

9. If the program ran to its end, `db` has finished too. If it's still stopped, leave the debugger with `q`.

Questions

1. ✎ What's a disassembler for, if you already have an assembler?

2. ✎ `dis` can't always tell code from data. Why not? What does it do about a byte that isn't an instruction?

3. ✎ What's the difference between `s` and `n` when the next instruction is a `jsr`?

4. ✎ What is a breakpoint? How does `db` make the program stop there? (Section 29.11.)

5. ✎ You find a bug with the debugger, and fix it by writing into memory. What must you still do?

Check yourself

☐	I CAN
☐	disassemble a program, with and without its labels
☐	start a program in the debugger, and read the line of registers
☐	step, set a breakpoint, and continue
☐	look at memory, and change it
☐	say what each of `s`, `n`, `c`, `b`, `r`, `m`, `d` and `q` does

CHALLENGE: ★ Section 29.8 hunts a real bug in `show1`. Section 29.12 shows how to make `show1.s` from Chapter 27's `show.s`. Make it, run `./show1 nothing`, and then find the bug yourself with `db` *before* you read how the manual found it.



PART THREE

Projects

A project is bigger than a lab: a program to type in and get going, and then to make your own. Each one starts with a listing that works. What you turn it into is up to you.

Take a project when you've done the labs it names. Each ends with a list of things to show when you've finished: a project isn't done until someone else has seen it run.

- Project 1. A Quiz
- Project 2. Turtle Pictures
- Project 3. A Round for Two Voices
- Project 4. Your Own Command
- Project 5. A Program in the Background
- Project 6. A Traffic Light
- Project 7. Faster, with Machine Code

PROJECT 1

A Quiz

THIS PROJECT: HyBASIC. After Labs 20 and 21. Two or three sittings. You'll use `DATA` and `READ`, `LINE INPUT`, `IF` blocks, strings and a score.

Name _____ Started _____ Finished _____

The starting point

Type this into `/sd/0/work/quiz.bas` with `edit`, and run it with `basic /sd/0/work/quiz.bas`.

```
' quiz.bas - a quiz: its questions and answers are in the DATA lines
SCORE = 0
READ COUNT
FOR Q = 1 TO COUNT
  READ QUESTION$, ANSWER$
  PRINT Q; QUESTION$
  LINE INPUT "   Your answer: "; REPLY$
  IF UCASE$(REPLY$) = UCASE$(ANSWER$) THEN
    PRINT "   Right!"
    SCORE = SCORE + 1
  ELSE
    PRINT "   No: it's "; ANSWER$; "."
  END IF
NEXT Q
PRINT
PRINT "You scored"; SCORE; "out of"; COUNT

DATA 4
DATA "What chip is the Hydra's processor?", "65C02"
DATA "How many tasks can a Hydra run at once?", "16"
DATA "CTRL and which key stops a program?", "C"
DATA "What is the Hydra's shell called?", "HyForth"
```

A run looks like this:

```
/ram> basic quiz.bas
 1 What chip is the Hydra's processor?
   Your answer: 65c02
   Right!
 2 How many tasks can a Hydra run at once?
   Your answer: 8
   No: it's 16.
 3 CTRL and which key stops a program?
   Your answer: c
   Right!
 4 What is the Hydra's shell called?
   Your answer: hyforth
   Right!

You scored 3 out of 4
```

Understand it first

1. ✎ The first `DATA` line holds just a 4. What's it for? What must you remember when you add a question?

2. ✎ Why does the program compare `UCASE$(REPLY$)` with `UCASE$(ANSWER$)`, and not just the two strings?

3. ✎ Why `LINE INPUT`, and not `INPUT`? (Try an answer with a comma in it, both ways.)

Make it yours

Do at least four of these. Tick each when it works.

☐	CHANGE
☐	Write your own questions, at least eight, about something you know well
☐	Ask the player's name at the start, and use it in the messages
☐	Give a second try before saying the answer
☐	Say something different at the end, depending on the score
☐	Let an answer be right if it's *in* the reply: "the 65C02" should count (`INSTR`)
☐	Ask the questions in a different order each time (`RND`, and an array)
☐	Keep the best score in a file, and say when it's been beaten (section 24.1)
☐	Take the time: how many seconds did the player need (`TIMER`)?

✎ Which did you do? What was the hardest part, and how did you get past it?

Show it

☐	BEFORE YOU CALL IT FINISHED
☐	Someone else has played it, without your help
☐	It copes with a silly answer, and with no answer at all
☐	The program has comments, and its lines are indented
☐	It's saved in `/sd/0/work`

CHALLENGE: ★ Take the questions out of the program altogether: keep them in a text file, one question and its answer to a line, so that a new quiz needs no programming. (`OPEN`, `LINE INPUT #` and `EOF`: section 24.1.)

PROJECT 2

Turtle Pictures

THIS PROJECT: HyForth, with a Vera X (the emulator has one). After Labs 16 and 17. Two sittings. You'll use `do` loops, words built from words, and the turtle of section 20.7.

Name _____ Started _____ Finished _____

The starting point

Imagine a turtle on the screen, with a pen tied to its tail. `fd` walks it forward, `rt` and `lt` turn it right and left, by a number of degrees, and wherever it walks, it draws. Type this into `/sd/0/work/shapes.fs`, and run it with `forth /sd/0/work/shapes.fs`.

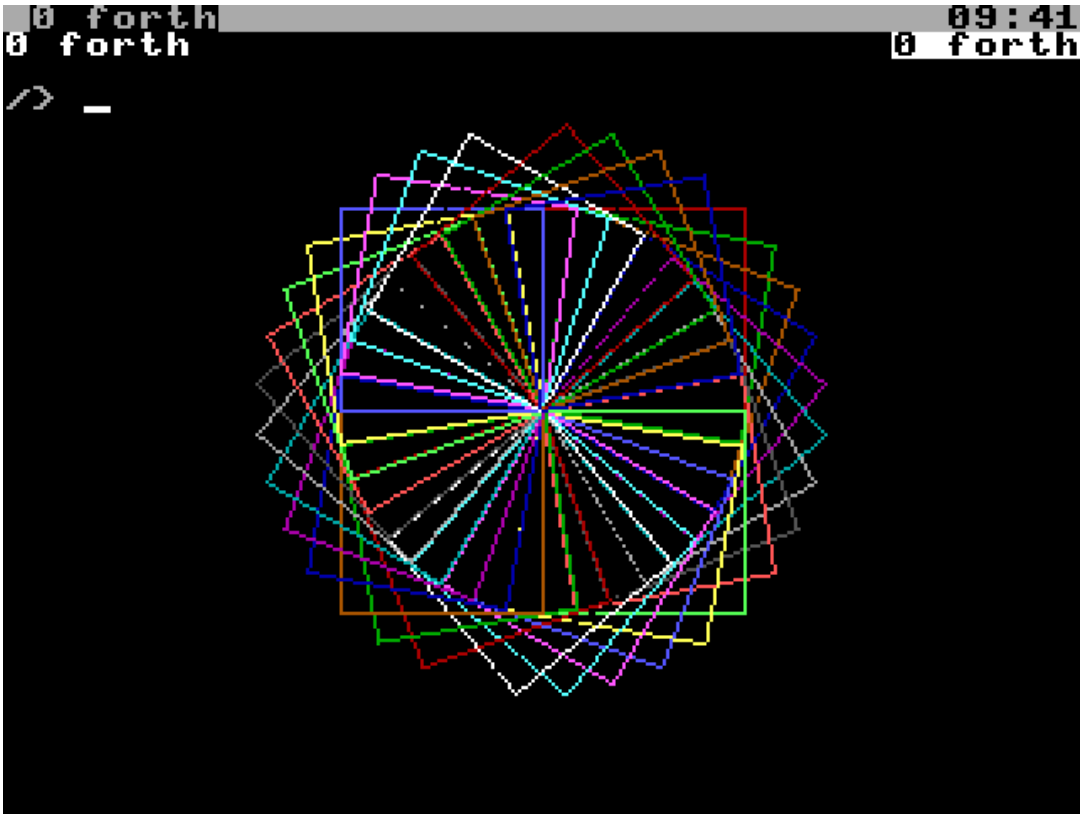
```
\ shapes.fs - turtle pictures (it needs a Vera X)
lib video

: polygon ( size sides -- )     \ A shape with that many sides, each that long
  360 over / swap               \ ( size turn sides )
  0 do over fd dup rt loop 2drop ;

: square   ( size -- ) 4 polygon ;
: triangle ( size -- ) 3 polygon ;

: flower ( -- )                 \ 36 squares, each turned a little, each a new colour
  36 0 do  i 15 mod 1+ pen  60 square  10 rt  loop ;

320 8 bitmap  cs
flower
```



Thirty-six squares make a flower.

To try words one at a time, start with `forth` alone, type `include /sd/0/work/shapes.fs`, and then things like `cs 80 triangle` at its prompt. (`cs` clears the screen, and puts the turtle back in the middle.)

Understand it first

1. ✎ To draw a square, the turtle turns 90 degrees at each corner. How far must it turn for a triangle? For a hexagon? What's the rule, and where is it in `polygon`?

2. ✎ Go through `polygon` a word at a time, for `60 4 polygon`, and write the stack after each word, up to the `do`.

3. ✎ What does `i 15 mod 1+ pen` do in `flower`? Why the `1+`?

Make it yours

☐	DRAW
☐	A row of polygons, from a triangle to an octagon (`pu` and `pd` lift and lower the pen)
☐	A star: five lines, turning 144 degrees each time
☐	A spiral: a line a little longer each time round the loop, with the same turn
☐	A circle made by the turtle: many tiny steps, with a tiny turn
☐	A flower made of some other shape than a square
☐	Your initials
☐	A picture of your own, built from at least three words you've defined

✎ Sketch your best picture, and write the words that drew it.

Show it

☐	BEFORE YOU CALL IT FINISHED
☐	Each word has a stack diagram, and leaves the stack as its diagram says
☐	The picture is drawn by one word, built from smaller ones
☐	Someone else has changed a number in your file, and guessed what would happen
☐	It's saved in `/sd/0/work`

CHALLENGE: ★ A spiral where the turn is 90 degrees makes a square spiral. What about 91? Or 120, 121, 144, 170? Write `spiral` (*turn* --), and explore. Which turns make stars, and why?

PROJECT 3

A Round for Two Voices

THIS PROJECT: Music, written as text. After Lab 12. Two sittings. You'll use MML and a score file (section 15.6), and you need sound: headphones, or the emulator's **Sound** button and **Real time** speed.

Name _____ Started _____ Finished _____

The starting point

A **round** is a tune that fits with itself: a second voice starts the same tune a little later, and the two sound well together. Type this into `/sd/0/work/frere.mml`, and play it with `play /sd/0/work/frere.mml`.

```
; frere.mml - Frere Jacques, as a round for two voices
#tempo 120
#title Frere Jacques
A I73 o4 l4 [c d e c]2 [e f g2]2 [g8 a8 g8 f8 e c]2 [c <g> c2]2
B I71 o4 l4 r1 r1 [c d e c]2 [e f g2]2 [g8 a8 g8 f8 e c]2 [c <g> c2]2
```

Understand it first

1. ✎ Lines `A` and `B` are almost the same. What's different, and what does that difference do?

2. ✎ What do these mean: `l4`, `g2`, `g8`, `[c d e c]2`, `<g>`, `r1`?

3. ✎ Each bar of this tune is four beats. Count the beats inside `[g8 a8 g8 f8 e c]`. Is it a whole bar?

4. ✎ `I73` and `I71` choose the instruments. Which are they? (Section 15.4.) Try two others.

Make it yours

☐	CHANGE
☐	Add a third voice, on line `C`, starting two bars after the second
☐	Make it faster, then slower (`#tempo`)
☐	Put one voice in the left speaker and one in the right (`p l` and `p r`)
☐	Make the second voice quieter (`v`)
☐	Add a drum on every beat, on another line (`x`, and section 15.4's list of drums)
☐	Write out a different round: "Row, Row, Row Your Boat", or "London's Burning"
☐	Write a tune of your own, at least eight bars, for two voices

✎ Write your own tune's lines here.

Show it

☐	BEFORE YOU CALL IT FINISHED
☐	Every bar adds up to the same number of beats, in every voice
☐	The file has a title, and a comment saying what it is
☐	Someone has listened to it all the way through
☐	It's saved in `/sd/0/work`

CHALLENGE: ★ Turn your score into a song file with `play -o`, and write a HyBASIC or hylang program that plays it in the background while it prints the words, a line at a time, in step with the music. (Chapter 24 or 22 has what you need; the sample program `jukebox` on the SD card does something like it, in C.)

PROJECT 4

Your Own Command

THIS PROJECT: rc, the shell's language. After Labs 5 and 6. One or two sittings. You'll write a script with arguments, keep it where the shell finds it, and so add a command to your Hydra.

Name _____ Started _____ Finished _____

The starting point

`note` is a notebook. `note` and some words adds a line, with the date and time in front. `note` alone shows the notebook. Type this into `/ram/note` with `edit`. (It keeps the notebook in `/sd/0/work`: make that directory first, if you haven't got one.)

```
#!/bin/rc
# note - a notebook.  note words ... adds a line, with the date; note alone shows them all
book=/sd/0/work/notebook
if(~ $#* 0) {
        cat $book
        exit
}
echo `{date} $* >>$book
```

Try it, as a script:

```
/ram> rc note Feed the cat
/ram> rc note Solder step 4
/ram> rc note
2000-01-01 00:00:35 Feed the cat
2000-01-01 00:00:37 Solder step 4
```

Then make it a command, by copying it into `/bin`. Now it runs by its name, from any directory:

```
/ram> cp note /bin/note
/ram> cd /
/> note Buy solder
/> note
```

Understand it first

1. ✎ What do `$*` and `$#*` stand for? What's `$#*` when you type just `note`?

2. ✎ What does `` `{date} `` do in the last line? What does >> do, and why not >?

3. ✎ You copied the script to `/bin`. Which directory is it really in? Will it be there tomorrow? Where should it go, so that it is? (Sections 9.7 and 14.2.)

Make it yours

☐	CHANGE
☐	Keep the command for good: put it where it will still be after the power's been off
☐	Say something sensible when there's no notebook yet, in place of `cat`'s error
☐	`note -n` shows how many notes there are (`wc -l`)
☐	`note -f word` shows only the notes with that word in them (`grep`)
☐	`note -l` shows only the last five (`tail`)
☐	`note -x` empties the notebook, but only after asking "Are you sure?"
☐	A wrong option prints a usage line, as the Hydra's own commands do

✎ Write your finished script here.

Show it

☐	BEFORE YOU CALL IT FINISHED
☐	It runs by its name, from any directory, after a reset
☐	Its first comment says what it does, and how to use it
☐	Someone else has used it without being told how, just from its usage line

CHALLENGE: ★ Write a second command of your own invention. Some ideas: `backup`, which copies everything in `/sd/0/work` into a directory named after today's date; `where`, which says which directory a command's program is in; `count`, which counts the files in each directory it's given.

PROJECT 5

A Program in the Background

THIS PROJECT: rc, and the Hydra's tasks. After Labs 4 and 6. One or two sittings. You'll start a program that keeps running while you do other things: the thing the Hydra-16 is named for.

Name _____ Started _____ Finished _____

The starting point

`ticker` writes the time into a file every five seconds, for ever. Type it into `/ram/ticker`.

```
#!/bin/rc
# ticker - write the time into a file every five seconds, until it's stopped
while(~ 1 1) {
        date >>/ram/ticker.log
        sleep 5
}
```

A `&` after a command starts it **in the background**: the shell doesn't wait for it, and you get your prompt back at once.

```
/ram> rc ticker &
/ram> ps
/ram> cat /ram/ticker.log
/ram> ls -l /ram
/ram> cat /ram/ticker.log
```

✎ Copy the lines of `ps` that belong to your ticker. There are two. What's each one doing?

Stop it with `kill` and the number of its `rc` task, and check with `ps` that both lines have gone.

Understand it first

1. ✎ While `ticker` runs, you can still type commands. How can one processor do both?

2. ✎ Most of the time, your ticker's tasks are in the state `sleep` or `wait`. How much of the processor's time do you think they use? Look at the `cpu` column of `ps` to check.

3. ✎ What happens to the ticker if you press CTRL-C at the prompt? If you reset the Hydra?

Make it yours

☐	CHANGE
☐	Take the delay as an argument: `ticker 2` ticks every two seconds
☐	Make it stop by itself after a number of ticks
☐	Watch it from a second window (Lab 8), with `tail /ram/ticker.log`
☐	Make it record something more interesting each time: the free space (`df`), or how many tasks are running
☐	Run two tickers at once, writing to the same file. Do their lines ever get mixed up?
☐	Write an alarm: `alarm 60 Tea is ready` waits that many seconds, then rings the bell and prints the message

✎ What did you find when two tasks wrote to one file? (The sample program `chorus`, from section 6.8, is about exactly this.)

Show it

☐	BEFORE YOU CALL IT FINISHED
☐	You can start it, find it with `ps`, and stop it, without looking anything up
☐	It doesn't fill the disk if it's left running (how do you know?)
☐	You can explain to someone what `&` does, and what a task is

CHALLENGE: ★ Run `/sd/0/sample/c/race`. Four tasks add to one counter, and lose count. Explain, in your own words and with a drawing, how two tasks can both add 1 and the counter only go up by 1. Then explain what the lock changes.

PROJECT 6

A Traffic Light

THIS PROJECT: rc, and the I/O header. After Lab 14. Two sittings. You need a real Hydra, three LEDs (red, amber and green), three 2.2K resistors, and wire. ⚠ Read the caution in section 17.1, and wire with the power off. (No real Hydra? See the last line of this page.)

Name _____ Started _____ Finished _____

The circuit

Each LED goes from +5 V, through its own resistor, through the LED, to a pin: red to PA2, amber to PA3, green to PA4. Wired this way round, an LED lights when its pin is **0**.

✎ Draw your circuit. Mark the header's pin numbers (the table in section 17.1), and which leg of each LED goes which way.

☐	BEFORE THE POWER GOES ON
☐	Each LED has its own resistor
☐	Nothing touches pins 3 and 4 of the header (PA0 and PA1: they're the I²C bus)
☐	Someone else has checked the wiring against the drawing

The starting point

Type this into `/ram/lights`, and run it with `rc /ram/lights`. CTRL-C stops it.

```
#!/bin/rc
# lights - a traffic light on pins 2 (red), 3 (amber) and 4 (green).  An LED lights when its pin is 0
fn on { echo 0 >/dev/gpio/$1 }
fn off { echo 1 >/dev/gpio/$1 }
for(p in 2 3 4) { echo out $p >/dev/gpio/ctl; off $p }
while(~ 1 1) {
        on 2; sleep 4
        on 3; sleep 1
        off 2; off 3; on 4; sleep 4
        off 4; on 3; sleep 1
        off 3
}
```

Understand it first

1. ✎ `on` and `off` are functions. What does `on 2` do? Why does `on` write a 0?

2. ✎ What does the `for` line do, before the lights start? Why is it needed?

3. ✎ Write out the order of the lights, with how long each lasts, from the `while` loop.

Make it yours

☐	CHANGE
☐	Use the timings and the order of the traffic lights where you live
☐	When the script is stopped, some lights stay on. Write `lightsoff`, to put them all out
☐	Add a push button on CA1 (section 17.2): the lights stay green until it's pressed, then go through red
☐	Add a second set of lights, on PA5, PA6 and PA7, for the road that crosses
☐	Run the lights in the background, and control them from the prompt
☐	Rewrite it in HyForth, hylang or HyBASIC, with that language's pin words

✎ Write your finished script here.

Show it

☐	BEFORE YOU CALL IT FINISHED
☐	It runs for five minutes without a mistake in the order
☐	You can explain why each LED needs a resistor, and why it lights on a 0
☐	The lights are never green both ways at once (if you built the crossroads)

CHALLENGE: ★ Everything about the lights' order and timing is buried in the `while` loop. Rewrite the script so that the order comes from a list at the top: each item a set of pins to light, and a number of seconds. Then a new sequence needs no new code. (No real Hydra? Do the whole project on the screen: draw three circles on the Vera X's bitmap, or print three coloured blocks with escape sequences, in place of the LEDs.)

PROJECT 7

Faster, with Machine Code

THIS PROJECT: HyBASIC with 65C02 assembly language inside it. After Labs 21, 24 and 25. Two sittings. You'll write the slow part of a program in the processor's own language, and measure what that buys.

Name _____ Started _____ Finished _____

The starting point

HyBASIC can hold assembly language between `ASM` and `END ASM` (section 27.20). This program counts to ten thousand twice: once in HyBASIC, and once in machine code. Type it into `/sd/0/work/fast.bas`.

```
' fast.bas - the same job in HyBASIC and in machine code: count to 10,000
T = TIMER
C = 0
FOR I = 1 TO 10000 : C = C + 1 : NEXT
PRINT "HyBASIC counted to"; C; "in"; TIMER - T; "seconds"

ASM
count:  ldx #0          ; X and Y together: a 16-bit count
        ldy #0
again:  inx             ; Add 1 to the low byte ...
        bne skip
        iny             ; ... and when it goes round to 0, to the high byte
skip:   cpx #$10        ; 10,000 is $2710: stop when X is $10 and Y is $27
        bne again
        cpy #$27
        bne again
        rts
END ASM

T = TIMER
CALL ASM count
RREG A, X, Y, P
PRINT "Machine code counted to"; Y * 256 + X; "in"; TIMER - T; "seconds"
```

```
/ram> basic fast.bas
HyBASIC counted to 10000 in 2.725 seconds
Machine code counted to 10000 in 0.055 seconds
```

Understand it first

1. ✎ How many times faster is the machine code, on your Hydra?

2. ✎ The 65C02's registers hold 8 bits, and 10,000 doesn't fit in 8 bits. How does the routine count that high? What are X and Y, together?

3. ✎ Run the routine by hand for its first three times round. When does `iny` first happen?

4. ✎ Why is `$2710` the number to stop at? How does the routine test for it? Why does it check X *and* Y?

5. ✎ What does `RREG A, X, Y, P` do? How does the HyBASIC line after it turn X and Y back into one number?

Make it yours

☐	CHANGE
☐	Count to 50,000 in both. Predict both times first
☐	Pass the number to count to *into* the routine: `CALL ASM count, 0, LOW, HIGH` puts values in X and Y
☐	Write a routine that prints a row of *X* stars (section 27.20 has one), and use it to draw a triangle
☐	Write a routine that adds up the numbers 1 to *X* in A, and check it against HyBASIC for X = 10
☐	Find the biggest X that routine gets right. What goes wrong after that, and which flag would tell you?

✎ Write one of your routines here, with a comment on every line.

Show it

☐	BEFORE YOU CALL IT FINISHED
☐	Every routine has been run by hand, on paper, for a small case
☐	Both versions give the same answer, and you've timed them
☐	You can say what each register is used for in your routine

CHALLENGE: ★ The race in section 25.6 gave HyForth about a fifth of a second for this job, and machine code is faster still. But HyBASIC took seconds to *write*, and the routine took much longer. For each of these, say which you'd use, and why: a loop that runs once when a program starts; a loop that draws every dot of a picture; a program you'll run once and throw away.



PART FOUR

Reference Cards

Single sheets to keep open beside the keyboard. Each one is the short version of a chapter of the manual: enough to jog your memory, with the section to turn to when it doesn't.

- Card 1. Keys
- Card 2. Everyday Commands
- Card 3. rc
- Card 4. HyForth
- Card 5. hylang
- Card 6. HyBASIC
- Card 7. The 65C02
- Card 8. Numbers and Memory

CARD 1

Keys

Typing a line (section 6.3)

KEYS	WHAT THEY DO
RETURN	Send the line
BACKSPACE	Rub out the letter before the cursor
← →	Move along the line
HOME or CTRL-A, END or CTRL-E	The start of the line; its end
CTRL-U	Rub out the whole line
↑ ↓	Bring back the lines typed before

At any time

KEYS	WHAT THEY DO
CTRL-C	Stop the program that's running
CTRL-D	At the start of a line: "that's the end of my typing"
CTRL-]	The window key: then one of the keys below

The window key, CTRL-], and then (section 10.3)

KEY	WHAT IT DOES	KEY	WHAT IT DOES
C	A new group, with a shell	W	The list of windows
0 to 9	Show that window	?	The window keys
TAB	The group's next window	N, P	The next group; the one before
An arrow	The window that way	X	Close the group shown

Where am I? (section 6.4)

THE PROMPT	WHO'S LISTENING	TO GET BACK
`/>`	HyForth, the shell	
Spaces	HyForth, in a definition	`;`
`%`	rc	`exit`
`>`	HyBASIC	`SYSTEM`
`hylang>`	hylang	`(exit)`
`*`	`ed`	`q`

THE PROMPT	WHO'S LISTENING	TO GET BACK
`db>`	The debugger	`q`

In the screen editor, `edit` (Chapter 11)

KEYS	WHAT THEY DO	KEYS	WHAT THEY DO
CTRL-O, RETURN	Save	CTRL-K	Cut the line
CTRL-X	Close the file, and leave	CTRL-U	Paste
CTRL-G	Show all the keys	ESC U	Undo
CTRL-W	Find	ESC G	Go to a line
CTRL-R	Open another file	ESC R	Find and replace

In the emulators (Chapter 2, Appendix H)

KEYS	WHAT THEY DO
CTRL-A then X	The terminal emulator, and the PC tool: quit
CTRL-A then R	The terminal emulator: press reset
CTRL-A twice	Send a real CTRL-A to the Hydra
CTRL-SHIFT-V	The web emulator: paste

CARD 2

Everyday Commands

Chapter 11 has every command, with its options. These are the ones you'll use most.

Looking

COMMAND	WHAT IT DOES
`ls` [`-l`] [*dir*]	List a directory (`-l`: with sizes and dates)
`pwd`	Say which directory you're in
`cd` [*dir*]	Go to a directory (`cd ..`: up one; `cd /`: to the top)
`cat` *file*	Show a file
`more` *file*	Show it a screenful at a time (Q leaves)
`head -5` *file*, `tail -5` *file*	Its first five lines; its last five
`xd` *file*	Its bytes, in hex

Changing

COMMAND	WHAT IT DOES
`edit` *file*	Make or change a file, in the screen editor
`cp` *from to*	Copy a file
`mv` *from to*	Move it, or rename it
`rm` *file*	Remove it, for good
`mkdir` *dir*, `rmdir` *dir*	Make a directory; remove an empty one
`echo` *words* `>`*file*	Put a line in a file (`>>`: add a line to its end)

Finding out

COMMAND	WHAT IT DOES
`wc` [`-l`] *file*	Count its lines, words and bytes (`-l`: just lines)
`grep` *word file*	Show the lines with a word in them (`-n`: with line numbers; `-c`: just count them)
`sort` [`-r`] *file*	Its lines in order (`-r`: backwards)
`cmp` *file1 file2*	Say whether two files are the same
`df`, `du` *dir*	How full the disks are; how much a directory holds
`date`	The date and time
`free`	How much memory there is
`calc '`*sum*`'`	Work out a sum: `calc '2^64'`, `calc -b x 255`

Tasks

COMMAND	WHAT IT DOES
`ps`	List the tasks
`top`	Show the busiest tasks, again and again (Q leaves)
command `&`	Start a command in the background
`kill` *task*, `slay` *name*	Stop a task by its number; by its name
`sleep` *seconds*	Wait

Languages and tools

COMMAND	WHAT IT DOES
`basic` [*file*], `hylang` [*file*], `forth` [*file*]	Start a language, or run a program in it
`rc` [*file*]	Start rc, or run a script
`play` *song*, `play -m 0 '`*mml*`'`	Play a song file; play a line of music
`as` *file*`.s`, `dis` *program*, `db` *program*	Assemble; disassemble; debug

Joining commands (Chapter 8)

WRITTEN	MEANS
command `>`*file*	Send its output to a file
command `<`*file*	Take its input from a file
command `|` *command*	Send the first one's output into the second (six commands at most)
command `&&` *command*	Do the second only if the first worked
command `||` *command*	Do the second only if the first failed

Where things are (section 6.5)

DIRECTORY	WHAT'S THERE
`/sd/0`	The SD card: **keep your work here**, in `/sd/0/work`
`/ram`	A RAM disk: quick, and emptied at every reset
`/rom`	HydraOS's own files: read, but never changed
`/bin`	Every command
`/dev`	The devices, as files

CARD 3

rc

The command language (Chapter 8). At the `/>` prompt, a line that doesn't start with a Forth word goes to rc; `%` and a space in front makes sure of it; and `rc` starts rc itself, with the prompt `%`.

Words and quotes

WRITTEN	MEANS
`'it''s here'`	One word, exactly as written (two `'` inside for one)
`a^b`	The two joined: `$name^.txt`
`# text`	A comment
command `;` *command*	One after the other
`{` *commands* `}`	Several commands as one

Variables

WRITTEN	MEANS
`x=hello`, `x=(a b c)`	Give a variable a word, or a list of words
`$x`	Its value: every word of it
`$x(2)`	Its second word
`$#x`	How many words it has
`$"x`	All its words as one word
`$*`, `$1`, `$#*`, `$0`	In a script or function: its arguments; the first; how many; its name
`$status`	How the last command ended: empty means it worked
`` `{ `` *command* `}`	The command's output, as words

Files and patterns

WRITTEN	MEANS
>*file*, >>*file*, <*file*	Output to a file; added to its end; input from one
`>[2]`*file*	Error messages to a file (`/dev/null` throws them away)
`*`	Any characters: `*.bas`
`?`	Any one character
`[abc]`, `[a-z]`	Any one of these

Decisions and loops

```
if(~ $x hello) echo yes          # ~ tests a word against patterns
if not echo no                   # the "else" of the if just before

for(f in *.txt) echo $f          # once for each word

while(~ $x hello) { ... }        # as long as the command succeeds

switch($x){
case h*
        echo starts with h
case *
        echo something else
}
```

WRITTEN	MEANS
`~` *word pattern ...*	Succeeds if the word matches any of the patterns
`!` *command*	Succeeds if the command fails
a `&&` *b*, *a* `||` *b*	Do *b* if *a* worked; if *a* failed

Functions and scripts

```
fn greet { echo Hello, $1! }     # a new command, until rc ends

#!/bin/rc                        # a script: a file of commands.
echo I was given $#* words: $*   # Run it with rc file, or copy it to /bin
```

Built in (section 8.14)

COMMAND	WHAT IT DOES
`cd` *dir*	Change directory
`exit`	Leave rc
`whatis` *name*	Say what a name is: a program, a function, a variable
`.` *file*	Run a file's commands in this rc, keeping what they set
`bind`, `mount`, `unmount`	Change the namespace (Chapter 9)
`wait`	Wait for background commands to end

CARD 4

HyForth

Chapter 19 teaches it, and Appendix A lists every word. The stack diagrams read (*before* -- *after*), with the top of the stack on the right.

The stack

WORD	STACK	WORD	STACK
`dup`	(*a* -- *a a*)	`swap`	(*a b* -- *b a*)
`drop`	(*a* --)	`over`	(*a b* -- *a b a*)
`rot`	(*a b c* -- *b c a*)	`nip`	(*a b* -- *b*)
`2dup`	(*a b* -- *a b a b*)	`2drop`	(*a b* --)
`.s`	Shows the stack	`depth`	(-- *how many*)

Arithmetic and tests

WORDS	WHAT THEY DO
`+ - * / mod`	(*a b* -- *c*): `7 2 mod` is 1
`/mod`	(*a b* -- *remainder quotient*)
`1+ 1- 2* 2/ negate abs`	(*a* -- *b*)
`min max`	(*a b* -- *c*)
`= <> < >`	(*a b* -- *flag*): -1 is true, 0 is false
`0= 0< 0>`	(*a* -- *flag*)
`and or xor invert`	On flags, and on bits

Printing

WORD	WHAT IT DOES
`.`	(*n* --) Print a number
`4 .r`	(*n* --) Print it in a column 4 wide
`."` *text*"	Print the text (in a definition)
`cr`, `space`, `5 spaces`	A new line; a space; five
`emit`	(*code* --) Print a character: `65 emit`, `[char] * emit`
`s"` *text*", `type`	(-- *addr len*) A string; (*addr len* --) print one

Definitions

```
: square ( n -- n*n ) dup * ;          \ a new word: ( ... ) and \ are comments
variable score      10 score !   score @ .   5 score +!
42 constant answer
0 value lives       3 to lives
create row 10 , 20 , 30 ,             \ row 1 cells + @  is 20
```

Decisions and loops (only inside a definition)

```
: sign ( n -- ) 0< if ." negative" else ." not negative" then ;
: five ( -- ) 5 0 do i . loop ;                    \ 0 1 2 3 4
: evens ( -- ) 10 0 do i . 2 +loop ;               \ 0 2 4 6 8
: down ( n -- ) begin dup . 1- dup 0= until drop ; \ n ... 1
```

`?do` for a loop that might go round no times; `leave` to leave one early; `exit` to leave a word; `recurse` for a word to call itself.

Files, libraries, and looking around

WORD	WHAT IT DOES
`include` *file*	Read a file of Forth, as if it were typed
`lib` *name*	Load a library: `random`, `facility`, `numbers`, `video`, `sound`, `hydra` ...
`words`	List every word
`see` *word*	Show a word's definition
`forth`, `bye`	A plain HyForth, where every line is Forth; leave it

When it goes wrong (section 19.15)

YOU SEE	IT MEANS
word `?`	HyForth doesn't know that word. A missing space?
`rc:` *word*`: not found`	The line's first word wasn't Forth, so rc had the line
A prompt of spaces	A `:` with no `;` yet
`stack underflow`	A word wanted more numbers than there were
`if: compile only`	That word only works inside a definition

CARD 5

hylang

Chapter 21 teaches it, and Appendix B lists every function. Start it with `hylang`; leave with `(exit)`; run a file with `hylang file.hl`.

The shapes of things

WRITTEN	IS
`(f a b)`	A call: work out `a` and `b`, then call `f` with them
`{a b c}`	A list, left as it is: data, or code to run later
`[a b c]`	A list of the *values* of `a`, `b` and `c`
`42`, `2/3`, `1.5`, `#xFF`	Numbers: whole, fractions, decimals, in other bases
`"text"`	A string (`\n` inside is a new line)
`:name`	An atom: a name that stands for itself
`T`, `NIL`	True; false (and "nothing", and the empty list)
`; text`	A comment

Names and functions

```
(def {x} 10)                      ; a global name          short: :(x 10)
(fun {square n} {* n n})          ; a named function
(fn {n} {* n 2})                  ; a function with no name  short: @({n} {* n 2})
(let {{a 1} {b 2}} (+ a b))       ; names for one expression
(do =(a 5) (print a) (* a a))     ; several things in turn: =( makes a name of this function's own
(set! {x} 11)                     ; change a name where it was made
```

Numbers and tests

FUNCTIONS	WHAT THEY GIVE
`(+ a b ...)` `(- a b)` `(* a b ...)` `(/ a b)`	Exact answers: `(/ 10 4)` is `5/2`
`(mod a b)`, `(div a b)`	The remainder; the whole part
`(sqrt x)`, `(pow x y)`, `(abs x)`	
`(to-fixed x 2)`, `(truncate x)`, `(round x)`	A decimal to 2 places; the whole part; the nearest whole number
`(random n)`	A whole number from 0 to $n - 1$, by chance
`(== a b)` `(!= a b)` `(< a b)` `(> a b)` `(<= a b)` `(>= a b)`	`T` or `NIL`
`(and a b)` `(or a b)` `(not a)`	

Decisions and loops

```
(if (> x 5) "big" "small")
(cond {(< x 0) "negative"} {(== x 0) "zero"} {T "positive"})
(dotimes {i 3} (print i))                    ; 0 1 2
(each {x {1 2 3}} (print x))
(while (< x 20) (set! {x} (+ x 1)))
(fun {fact n} {if (< n 2) 1 (* n (fact (- n 1)))})    ; a function that calls itself
```

Lists

FUNCTIONS	WHAT THEY GIVE
`(len l)`, `(fst l)`, `(last l)`, `(nth 2 l)`	How many; the first; the last; item 2, counting from 0
`(head l)`, `(tail l)`, `(cons x l)`, `(join l m)`	A list of the first; the rest; *x* in front; two joined
`(range 5)`, `(range 1 11)`	`{0 1 2 3 4}`; `{1 2 ... 10}`
`(map f l)`, `(filter f l)`	*f* of every item; the items *f* is true of
`(sum l)`, `(sort l)`, `(reverse l)`, `(elem? x l)`	

Strings, hashes, printing

FUNCTIONS	WHAT THEY DO
`(+ "a" "b" 3)`	Join: `"ab3"` (a string must come first)
`(len s)`, `(substring s 0 5)`, `(str-upper s)`	
`(str-split s " ")`, `(str-join l " ")`	A string to a list of words, and back
`(format "{} is {}" a b)`	Fill each `{}` with the next value
`(val s)`, `(to-str x)`	A string to a number; anything to a string
`(def {h} #({{:name "Rex"} {:legs 4}}))`	A hash: values under names
`(h :name)`, `(hash-put h {:legs 3})`, `(hash-keys h)`	Look up; change; the names
`(print a b)`, `(write a b)`	Print with a new line; without
`(read-line)`	A line typed at the keyboard, as a string

When it goes wrong (section 21.13)

YOU SEE	IT MEANS
`Unbound Symbol 'x'`	A name hylang doesn't know: a spelling slip?
A prompt of spaces	A `(`, `{` or `"` isn't closed yet
`<function>` for an answer	Too few arguments
`Unbound Symbol 'n'`, defining a function	The body is in `( )`: it should be in `{ }`

CARD 6

HyBASIC

Chapter 23 teaches it, and Appendix C lists every keyword. Start it with `basic`; leave with `SYSTEM`; run a file with `basic file.bas`.

At the prompt

TYPED	WHAT IT DOES
`PRINT 2 + 3` (or `? 2 + 3`)	Do a line at once
`10 PRINT "Hi"`	Keep a line, as line 10 of the program
`10`	Remove line 10
`LIST`, `RUN`, `NEW`	Show the program; run it; throw it away
`SAVE "file.bas"`, `LOAD "file.bas"`	Keep it in a file; bring it back
`EDIT`	Change it in the screen editor
CTRL-C, then `CONT`	Stop a running program; carry on

Values

WRITTEN	IS
`SCORE = 10`	A number variable
`NAME$ = "Sam"`	A string variable: its name ends in `$`
`DIM A(10)`	An array: `A(0)` to `A(10)`
`+ - * /`, `\`, `MOD`, `^`	Arithmetic: `/` is exact (`10 / 4` is `5/2`); `\` keeps the whole part
`= <> < > <= >=`, `AND OR NOT`	Tests: -1 is true, 0 is false
`' text`, `REM text`	A comment

Printing and asking

```
PRINT "Total:"; T          ' ; keeps things together   , moves to the next column
PRINT USING "###.##"; X    ' in a fixed shape
INPUT "How many"; N        ' ask for a number
LINE INPUT "Name: "; N$    ' ask for a whole line of text
```

Decisions

```
IF A > 3 THEN PRINT "big" ELSE PRINT "small"

IF SCORE >= 90 THEN
  PRINT "Excellent!"
ELSEIF SCORE >= 50 THEN
  PRINT "Well done."
ELSE
  PRINT "Try again."
END IF

SELECT CASE D
  CASE 0, 6 : PRINT "weekend"
  CASE ELSE : PRINT "weekday"
END SELECT
```

Loops

```
FOR I = 1 TO 10 STEP 2 : PRINT I : NEXT I

DO WHILE N < 100           ' tested first: may not run at all
  N = N * 2
LOOP

DO                         ' tested last: runs at least once
  INPUT "Guess"; G
LOOP UNTIL G = SECRET
```

`EXIT FOR` and `EXIT DO` leave a loop early.

Strings and numbers

FUNCTION	GIVES
`LEN(S$), LEFT$(S$, 3), RIGHT$(S$, 3), MID$(S$, 2, 3)`	Its length; its first 3; its last 3; 3 from the 2nd
`UCASE$(S$), LCASE$(S$), INSTR(S$, "x")`	Capitals; small letters; where `x` is (0: nowhere)
`STR$(N), VAL(S$), CHR$(65), ASC("A")`	Number to string; string to number; code to character; and back
`INT(X), ABS(X), SQR(X), FIXED(X, 2)`	Rounded down; its size; its square root; as a decimal to 2 places
`RND`, `INT(RND * 6) + 1`, `RANDOMIZE TIMER`	0 to 1 by chance; a die; shake the dice

Data and procedures

```
READ N$, AGE               ' takes the next values from the DATA lines
DATA "Sam", 12

SUB GREET (WHO$)           ' a procedure: GREET "Sam"
  PRINT "Hello, "; WHO$
END SUB

FUNCTION SQUARE (N)        ' a function: PRINT SQUARE(7)
  SQUARE = N * N
END FUNCTION
```

A `SUB`'s or `FUNCTION`'s variables are its own (section 23.12): `SHARED` names the main program's that it uses.

When it goes wrong (section 23.13)

YOU SEE	IT MEANS
`syntax error`	A word spelled wrong, or a quote or parenthesis missing

YOU SEE	IT MEANS
`type mismatch`	A string where a number should be, or the other way
`?Redo from start`	`INPUT` wanted a number
A 0 you didn't expect	A variable's name spelled two ways

CARD 7

The 65C02

Chapter 26 teaches it, and Appendix D has every instruction's opcodes and timings.

The registers

REGISTER	BITS	WHAT IT'S FOR
A	8	The accumulator: arithmetic and logic happen here
X, Y	8	Index registers: counting, and pointing into tables
S	8	The stack pointer: the stack is at `$0100` to `$01FF`
PC	16	The program counter: the next instruction's address
P	8	The flags: N V - B D I Z C

The flags

FLAG	IS 1 WHEN
N	The last result was negative (its top bit was 1)
Z	The last result was zero
C	An addition carried out of the byte; or a subtraction *didn't* borrow; or a shift pushed a 1 out
V	A signed sum went out of range
I	Interrupts are held off
D	`ADC` and `SBC` work in decimal

The instructions, by job

JOB	INSTRUCTIONS
Load and store	`LDA LDX LDY STA STX STY STZ` (store zero)
Copy between registers	`TAX TAY TXA TYA TSX TXS`
The stack	`PHA PLA PHX PLX PHY PLY PHP PLP`
Add and subtract	`ADC SBC` (both use the carry: `CLC` before adding, `SEC` before subtracting)
Count	`INC DEC INX DEX INY DEY`
Logic	`AND ORA EOR BIT TRB TSB`
Shift	`ASL LSR ROL ROR`
Compare	`CMP CPX CPY` (they set Z, C and N, and change nothing else)
Branch	`BEQ BNE` (on Z); `BCS BCC` (on C); `BMI BPL` (on N); `BVS BVC` (on V); `BRA` (always)
Jump and call	`JMP JSR RTS`

JOB	INSTRUCTIONS
Flags	`CLC SEC CLD SED CLI SEI CLV`
Interrupts and the rest	`BRK RTI NOP WAI STP`
Single bits	`RMB0-7 SMB0-7 BBR0-7 BBS0-7`

After a compare

AFTER `CMP #N`	BRANCH WITH
A equals *n*	`beq`
A differs from *n*	`bne`
A is *n* or more (unsigned)	`bcs`
A is less than *n* (unsigned)	`bcc`

The addressing modes (section 26.6)

WRITTEN	THE BYTE IT MEANS
`lda #$41`	The number `$41` itself
`lda $22`	The byte at `$0022`, on the zero page
`lda $1234`	The byte at `$1234`
`lda $1234,x`	The byte at `$1234` + X
`lda ($22)`	The byte at the address kept in `$22` and `$23`
`lda ($22),y`	The byte at that address + Y
`asl a`	A itself
`bne loop`	(A branch: up to 128 bytes back, or 127 on)

A program's skeleton, for `as` (Chapter 27)

```
.include "hydra.inc"            ; the system calls' names
.include "hyx2.inc"             ; HYX2_PROGRAM
.include "macros.inc"           ; LF, PRINT, LDR ...

            HYX2_PROGRAM "name", main

.zeropage
count:      .res 1              ; a variable

.code
main:       lda #'A'
            jsr PUTC            ; a system call: print the character in A
            rts                 ; the end
```

`as name.s` assembles it; `./name` runs it; `as -l` writes a labels file too; `dis name` takes it apart; `db ./name` runs it an instruction at a time (`s` step, `n` next, `c` continue, `b` breakpoint, `r` registers, `m` memory, `q` quit).

CARD 8

Numbers and Memory

Hex digits (section 1.6)

DECIMAL	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

A byte is two hex digits: `$A5` is `1010 0101`, which is 10 × 16 + 5 = 165.

Powers of two

POWER	DECIMAL	HEX	WORTH KNOWING
2^4	16	`$10`	One hex digit's worth
2^7	128	`$80`	A byte's top bit
2^8	256	`$100`	The values a byte can hold; a page
2^10	1,024	`$400`	1K
2^13	8,192	`$2000`	8K: a RAM bank
2^14	16,384	`$4000`	16K: a paged ROM bank
2^15	32,768	`$8000`	32K: a task's own memory
2^16	65,536	`$10000`	64K: all the 65C02 can see at once
2^20	1,048,576	`$100000`	1 MB

Writing a number in a base (Chapter 18)

WRITTEN	WHERE	MEANS
`$FF`	This book, assembly language, HyForth	Hex
`#xFF`, `#b1010`, `#o17`	`calc`, hylang, HyBASIC, HyForth's numbers library	Hex; binary; octal
`%1010`	HyForth, assembly language	Binary
`&HFF`	HyBASIC	Hex
`calc -b x 255`	The shell	Print 255 in hex

Characters (Appendix E has them all)

CODE	IS	CODE	IS	CODE	IS
10 (`$0A`)	A new line	48 to 57 (`$30`-`$39`)	`0` to `9`	65 to 90 (`$41`-`$5A`)	`A` to `Z`
27 (`$1B`)	ESC	32 (`$20`)	A space	97 to 122 (`$61`-`$7A`)	`a` to `z`

A small letter's code is its capital's plus 32. A digit's code is the digit plus 48.

The Hydra's memory, as the 65C02 sees it (section 7.2)

ADDRESSES	WHAT'S THERE	CHOSEN BY
`$0000-$7FFF`	The running task's own 32K of RAM	The T register
`$8000-$9FFF`	A RAM bank: 8K of a memory card, or of the shared RAM	The RAM bank register, at `$00`
`$A000-$DFFF`	A paged ROM bank: 16K	The ROM bank register, at `$01`
`$E000-$FEFF`	A page of the kernel ROM: 8K	The W register
`$FF00-$FFEF`	The I/O ports: the chips that talk to the world	
`$FFF0-$FFF3`	The registers T, U, V and W	
`$FFFE-$FFFF`	The vector RAM: where interrupts go	

Inside a task's 32K

ADDRESSES	WHAT'S THERE
`$00`, `$01`	The RAM and ROM bank registers
`$02-$21`	The call registers, r0 to r15: two bytes each, for system calls
`$22-$7F`	The program's own zero page: 94 bytes for its variables
`$80-$FF`	HydraOS's zero page for the task
`$0100-$01FF`	The stack
`$0200-$03FF`	HydraOS's notes on the task: its open files, its name, its arguments
`$0800` on	A program: its header (48 bytes), then its code, its data and its free memory

Sizes

THING	SIZE
A bit	One 0 or 1
A byte	8 bits: 0 to 255
A HyForth cell	16 bits: 0 to 65,535, or -32,768 to 32,767
An address	16 bits
1K	1,024 bytes
1 MB	1,024 K
A disk block	512 bytes
A tick	1/200 of a second

PART FIVE

Your Record

A logbook for what you did and what you found out, and a list of skills to tick off as you gain them. Engineers and scientists keep logbooks all their lives. Start now.

- The Logbook
- The Skills Passport

The Logbook

One line for each time you sit down at the Hydra. It takes a minute, and it's the first thing you'll want when you come back after a week away: where was I, and what was I about to try?

How to keep it

- **What I did**: the lab, the project, or the part of the build.
- **What I found out**: one thing you didn't know before. A surprise is best.
- **Next time**: the very next thing to do. Write it while you still remember.

DATE	WHAT I DID	WHAT I FOUND OUT	NEXT TIME

DATE	WHAT I DID	WHAT I FOUND OUT	NEXT TIME

Things I want to try

Ideas come at awkward moments. Write them here, and come back to them.

Things that went wrong, and why

The most useful page in any logbook. A mistake you've written down, you'll only make once.

DATE	WHAT WENT WRONG	WHY	HOW I'LL KNOW NEXT TIME

DATE	WHAT WENT WRONG	WHY	HOW I'LL KNOW NEXT TIME

The Skills Passport

Tick a skill when you can do it without looking it up, and get it signed by someone who watched you do it: a teacher, or a friend who can already do it (working alone? sign it yourself, honestly). The best way to be sure you can do something is to show someone else how.

Building

☐	I CAN	SIGNED
☐	work safely with a soldering iron	
☐	read a resistor's and a capacitor's value	
☐	make a good solder joint, and tell a bad one	
☐	repair a bridge, and take out a part	
☐	fit a chip the right way round, without bending its legs	
☐	measure a voltage and a resistance	
☐	find a fault by changing one thing at a time	

Using the Hydra

☐	I CAN	SIGNED
☐	start a Hydra or an emulator, and get to the prompt	
☐	type, fix and repeat a command	
☐	stop a program, and get out of any prompt	
☐	find my way round the files	
☐	make, change, copy and remove a file	
☐	keep my work where it's safe	
☐	open and use more than one window	
☐	look a command up in the manual, and use it	
☐	join commands with pipes, and send output to a file	
☐	write an rc script, and make it a command	
☐	start a program in the background, find it and stop it	

Devices

☐	I CAN	SIGNED
☐	read a device's control file, and send it a command	
☐	check a disk, and say how full it is	
☐	play a tune I've written	
☐	draw a picture with commands	
☐	switch a pin on and off (and light an LED with it)	
☐	change the screen's colours with an escape sequence	

Programming

☐	I CAN	SIGNED
☐	count in binary and hex	
☐	write and run a program in HyBASIC	
☐	write and use my own words in HyForth	
☐	write and use my own functions in hylang	
☐	use a loop, a decision and a variable in at least one language	
☐	keep a program in a file, and run it from the shell	
☐	read a program in a language I don't know well, and say what it does	
☐	find a mistake in a program by testing its small parts	
☐	explain a program of mine to someone else	

Machine language

☐	I CAN	SIGNED
☐	name the 65C02's registers and flags	
☐	follow a short machine-code program by hand	
☐	write, assemble and run a program in assembly language	
☐	read a disassembly	
☐	step through a program in the debugger	

Understanding

☐	I CAN EXPLAIN	SIGNED
☐	what the processor, the memory and the bus each do	
☐	how a Hydra runs several programs at once	
☐	what an operating system is for	
☐	what "everything is a file" means on the Hydra	
☐	why there are different programming languages	
☐	what happens between switching on and the prompt	

Finished

	DATE	SIGNED
I built a Hydra-16, and it works		
I finished the labs		
I finished a project of my own		